Irene Documentation

Release 1.2.5

Mehdi Ghasemi

Mar 13, 2026

CONTENTS

1 Introduction **3**
- 1.1 Documentation Scope . . . 3
- 1.2 Requirements and dependencies . . . 3
- 1.3 Download . . . 5
- 1.4 Installation . . . 5
- 1.5 License . . . 5

2 Architecture and Method Guide **7**
- 2.1 From Polynomial Expressions to Group-Rings . . . 7
- 2.2 Module Layers . . . 7
- 2.3 Choosing a Relaxation Family . . . 7
- 2.4 Typical Workflow . . . 8

3 Group-Ring Foundations **9**
- 3.1 Algebraic Setting . . . 9
- 3.2 Commutative Semigroup and Semigroup Algebra . . . 9
- 3.3 Support and Geometry . . . 9
- 3.4 Differential Operators . . . 9
- 3.5 Why This Matters for POP . . . 10

4 Problem Representation **11**
- 4.1 Problem Template . . . 11
- 4.2 Core Responsibilities . . . 11
- 4.3 Degree Conventions . . . 11
- 4.4 Key Methods . . . 11
- 4.5 From Equations to Methods . . . 12
- 4.6 Delta Sets and Newton Data . . . 12
- 4.7 Minimal Construction Pattern . . . 12

5 Semidefinite Programming **13**
- 5.1 Primal and Dual formulations . . . 13
- 5.2 The `sdp` class . . . 13

6 Optimization **17**
- 6.1 Notation Bridge to Geometric and SONC Chapters . . . 18
- 6.2 Beyond SDP: Alternative Lower-Bound Mechanisms . . . 19
- 6.3 Moment Constraints . . . 20
- 6.4 Equality Constraints . . . 21
- 6.5 Extracting solutions . . . 32

7 Geometric Programming Relaxations **37**

7.1 Theory to Implementation . . . 37
7.2 Interpretation of the Matrix Transform . . . 37
7.3 Constraint Families in `solve` . . . 38
7.4 Lower-Bound Meaning . . . 38
7.5 Practical Notes . . . 38

8 SONC Relaxations **39**
8.1 Formulation Focus . . . 39
8.2 Variable Roles . . . 39
8.3 Branching on $\lambda_0^{(\beta)}$ . . . 40
8.4 Implementation Stages in `SONCRelaxations` . . . 40
8.5 Theory-to-Code Anchors . . . 40
8.6 Geometric Interpretation . . . 40
8.7 Repository Anchors . . . 41

9 Approximating Optimum Value **43**
9.1 Using `pyProximation.OrthSystem` . . . 43

10 Benchmark Optimization Problems **53**
10.1 Rosenbrock Function . . . 53
10.2 Giunta Function . . . 57
10.3 Parsopoulos Function . . . 60
10.4 Shubert Function . . . 63
10.5 McCormick Function . . . 66
10.6 Schaffer Function N.2 . . . 67
10.7 Schaffer Function N.4 . . . 69
10.8 Drop-Wave Function . . . 70

11 Examples and Validation **73**
11.1 Recommended Example Sequence . . . 73
11.2 API Quick Reference . . . 73
11.3 SDP Example . . . 73
11.4 Geometric Programming Example . . . 74
11.5 SONC Examples . . . 74
11.6 Example 3.3 Traceability . . . 74
11.7 Regression Validation . . . 74

12 Code Documentation **75**

13 pyProximation (lean) **97**
13.1 Requirements and dependencies . . . 97
13.2 Download . . . 97
13.3 Installation . . . 97
13.4 License . . . 97

14 Measures **99**
14.1 Density function . . . 99
14.2 Integrals . . . 100
14.3 p-norms . . . 100
14.4 Drawing samples . . . 101

15 Hilbert Spaces **103**
15.1 Orthonormal system of functions . . . 103
15.2 OrthSystem . . . 104
15.3 Approximation . . . 106

15.4 Rational Approximation . . . 108

16 Interpolation **109**
16.1 Lagrange interpolation . . . 109
16.2 L^2-approximation with discrete measures . . . 110

17 Code Documentation **113**

18 Revision History **117**

19 To Do **119**
19.1 Done . . . 119

20 Appendix **121**
20.1 pyProximation . . . 121
20.2 pyOpt . . . 121
20.3 LaTeX support . . . 123

21 Indices and tables **125**

Bibliography **127**

Python Module Index **129**

Contents:

CHAPTER

ONE

INTRODUCTION

This is a brief documentation for using *Irene*. Irene was originally written to find reliable approximations for optimum value of an arbitrary optimization problem. It implements a modification of Lasserre's SDP Relaxations based on generalized truncated moment problem to handle general optimization problems algebraically.

1.1 Documentation Scope

The documentation now covers three complementary pathways for constrained polynomial optimization:

1. SDP hierarchy methods (moment/SOS based).
2. Geometric programming relaxations.
3. SONC relaxations.

The recommended reading path is:

1. `architecture` for conceptual map.
2. `algebra` and `program` for representation layer.
3. `sdp`, `geometric`, and `sonc` for method-specific formulations.

1.2 Requirements and dependencies

This is a python package, so clearly python is an obvious requirement. Irene relies on the following packages:

- **for vector calculations:**
  - NumPy.
  - SciPy.
- **for symbolic computations:**
  - SymPy.
- **for semidefinite optimization, at least one of the following is required:**
  - cvxopt,
  - dsdp,
  - sdpa,
  - csdp.

1.2.1 Dependency Matrix by Method Family

Method family • Core Python packages • Optional packages • External solver requirement
SDP relaxations • numpy, scipy, sympy • cvxopt • one of cvxopt, dsdp, sdpa, csdp
Geometric relaxations • numpy, scipy, sympy • gpkit • gpkit-supported GP backend
SONC relaxations • numpy, scipy, sympy • gpkit • gpkit-supported GP backend

1.2.2 Solver Prerequisites

Before running examples, verify available solvers from Python:

```
from Irene.base import base
print(base().AvailableSDPSolvers())
```

1.2.3 Quick Validation Workflow

After installation, the following commands provide a practical smoke test:

```
python examples/Example01.py
python examples/GPExample.py
python examples/SONCExample.py
python -m unittest discover tests/
```

1.2.4 Solver Troubleshooting

Common runtime signatures and first actions:

1. `AvailableSDPSolvers()` returns an empty list.

   This indicates that no configured SDP backend is currently reachable. Install at least one supported solver and verify it is available on `PATH` (or configured in solver path settings on platforms that require explicit paths).

2. `RuntimeError: GP solve failed` or `RuntimeError: SONC GP solve failed`.

   These messages usually indicate missing GP backend support, an unavailable solver, or an infeasible/numerically unstable relaxation for the selected formulation. Start with the shipped examples, lower verbosity, and simplified instances.

3. `ModuleNotFoundError: No module named 'gpkit'`.

   Install `gpkit` before running geometric or SONC examples.

4. SDP solve runs but returns non-optimal status.

Try another supported SDP solver, inspect constraints for scaling issues, and compare with a lower relaxation order before increasing model complexity.

1.3 Download

Irene can be obtained from https://github.com/mghasemi/Irene.

1.4 Installation

To install *Irene*, run the following in terminal:

```
sudo python setup.py install
```

1.4.1 Documentation

The documentation of *Irene* is prepared via sphinx.

To compile html version of the documentation run:

```
$Irene/doc/make html
```

To make a pdf file,subject to existence of `latexpdf` run:

```
$Irene/doc/make latexpdf
```

Documentation is also available at http://irene.readthedocs.io.

1.5 License

Irene is distributed under MIT license:

1.5.1 MIT License

Copyright (c) 2016-2026 Mehdi Ghasemi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

CHAPTER

TWO

ARCHITECTURE AND METHOD GUIDE

The current codebase supports multiple constrained polynomial optimization pathways. Historically, the documentation emphasized SDP hierarchies. The present structure extends this to include geometric and SONC methods on the same algebraic backbone.

2.1 From Polynomial Expressions to Group-Rings

Instead of treating input only as classical polynomial expressions, Irene builds optimization models through commutative semigroups and their semigroup algebras. This representation supports:

1. Symbolic manipulation of monomial support and exponents.
2. Geometric operations on Newton polytopes.
3. Differential operators through user-provided derivation maps.

The shift in perspective is important: instead of viewing optimization only as operations in $\mathbb{R}[x_1, \ldots, x_n]$, Irene treats expressions as elements of a semigroup algebra that can be converted to geometric data (supports, exponent tuples, convex combinations) used directly in GP and SONC routines.

2.2 Module Layers

1. Algebra layer: `grouprings.py`.
2. Problem layer: `program.py`.
3. Relaxation layer: `relaxations.py`, `geometric.py`, `sonc.py`.
4. Solver layer: `sdp.py` and external solver backends.

2.3 Choosing a Relaxation Family

Use SDP relaxations when you need a moment/SOS hierarchy and semidefinite certificates. Use geometric relaxations when the transformed formulation is naturally handled by GP. Use SONC relaxations for sparse circuit-structured formulations in constrained settings.

Table 1: Method Selection Snapshot

Method • Primary object • Typical strength • Main module
SDP hierarchy • Moment and localizing matrices • Certified lower bounds via SOS/moment conditions • `relaxations.py` and `sdp.py`
Geometric relaxation • Posynomial/signomial GP model • Scalable lower bounds for suitable transformed instances • `geometric.py`
SONC relaxation • Circuit polynomial constraints • Sparse constrained formulations with barycentric structure • `sonc.py`

2.4 Typical Workflow

1. Define algebra generators and build expressions in a semigroup algebra.
2. Form an optimization problem with objective and constraints.
3. Select relaxation family (SDP, GP, or SONC).
4. Solve and interpret a certified lower bound.

CHAPTER

THREE

GROUP-RING FOUNDATIONS

This chapter introduces the algebraic layer behind the optimization modules.

3.1 Algebraic Setting

Let S be a finitely generated commutative semigroup and let $\mathbb{R}[S]$ denote its semigroup algebra. In Irene, elements of $\mathbb{R}[S]$ are represented as finite sums

$$f = \sum_{\alpha \in \mathrm{supp}(f)} c_\alpha \alpha,$$

where α is a semigroup element and $c_\alpha \in \mathbb{R}$. This perspective generalizes classical polynomial notation and keeps support, degree, and structural operations explicit for optimization routines.

When relations are present, the algebra is effectively handled modulo those relations through reduction in the semigroup representation. This is useful for modeling quotient structures that arise naturally in symbolic formulations.

3.2 Commutative Semigroup and Semigroup Algebra

The module `grouprings.py` provides:

1. `CommutativeSemigroup`: generator-based semigroup representation.
2. `SemigroupAlgebra`: algebra construction over that semigroup.
3. `AtomicSGElement` and `SemigroupAlgebraElement`: monomial and polynomial-like elements.

This representation is used by higher layers to extract supports, exponents, and structural information required by geometric and SONC relaxations.

3.3 Support and Geometry

The support of f is central in both geometric and SONC constructions. By converting semigroup monomials to exponent tuples, Irene can compute Newton polytope information and barycentric relations directly from algebraic input.

This is the key bridge from symbolic algebra to convex-geometric objects used in lower-bound certificates.

3.4 Differential Operators

Beyond static algebraic representation, `SemigroupAlgebra` supports derivations. Given a base map, derivatives are propagated through expressions using product-rule behavior.

For factors u and v, the implementation follows:

$$D(uv) = D(u)v + uD(v).$$

This enables a workflow where optimization is not only over algebraic objects, but also over structures enriched with differential operators.

In code, the derivative path is organized as:

1. `SemigroupAlgebra.add_derivative` registers a derivation map.
2. `SemigroupAlgebra.derivative` selects a registered derivation.
3. `SemigroupAlgebra.diff` applies recursive product-rule expansion.

This method-level design makes differentiation explicit and extensible for problem formulations where algebraic structure and operator behavior are coupled.

From a theoretical viewpoint, derivations are linear maps $D : \mathbb{R}[S] \to \mathbb{R}[S]$ that satisfy Leibniz rules. Irene's `add_derivative` and `diff` pipeline implements this behavior directly on semigroup-algebra elements.

3.5 Why This Matters for POP

The shift from polynomial-only notation to group-ring structures improves the connection between symbolic representation and geometric computation:

1. Exponent vectors are directly accessible for convex hull and delta-set routines.
2. Algebraic reductions remain explicit and programmatic.
3. The same representation supports SDP, GP, and SONC method families.

CHAPTER

FOUR

PROBLEM REPRESENTATION

The class `OptimizationProblem` in `program.py` is the bridge between algebraic expressions and concrete relaxation models.

4.1 Problem Template

The optimization model is represented in algebraic form as:

$$\min f \quad \text{subject to} \quad g_i \geq 0,\ i = 1, \ldots, m,$$

where f and g_i are semigroup-algebra elements. This keeps the problem in a symbolic domain until a relaxation method converts it to SDP or GP data structures.

4.2 Core Responsibilities

1. Store objective and constraints as semigroup-algebra expressions.
2. Track degrees used by relaxation constructors.
3. Provide support analysis utilities used by geometric and SONC modules.

4.3 Degree Conventions

`OptimizationProblem` tracks objective and constraint degrees and uses an even program degree for relaxation construction. This convention matches the structure needed by polynomial nonnegativity certificates and by the GP/SONC term-partitioning routines.

In particular, the effective order used by downstream methods is the smallest even degree greater than or equal to the maximum polynomial degree in the problem.

4.4 Key Methods

`set_objective`
: Registers the objective and updates degree metadata.

`add_constraints`
: Adds constrained expressions and updates per-constraint degree metadata.

`delta`
: Extracts terms relevant for nonnegativity and circuit-based conditions.

`mono2ord_tuple` and `tuple2mono`
: Convert symbolic monomials to numeric exponent tuples and back.

`newton`, `convex_combination`, and related helpers
Support Newton-polytope and barycentric calculations required by SONC/GP paths.

4.5 From Equations to Methods

This chapter connects internal utilities in `OptimizationProblem` to the equation-level objects used in *Geometric Programming Relaxations* and *SONC Relaxations*.

Mathematical role	OptimizationProblem method	Where it appears later
Build active support and delta terms	`delta`	Section 4 GP construction and Section 3 constrained SONC families
Move between symbolic monomials and exponent tuples	`mono2ord_tuple` and `tuple2mono`	Newton-polytope geometry and constraint assembly in GP and SONC
Compute support geometry	`newton`	Support-point extraction for SONC and transformed GP structures
Compute barycentric coefficients	`convex_combination` and `linear_combination`	SONC weights $\lambda^{(\beta)}$ and related support relations

In practice, these methods are the bridge from symbolic semigroup-algebra input to the numeric data structures used by the geometric and SONC solvers.

4.6 Delta Sets and Newton Data

The `delta` and `newton` family of methods provide the two core theoretical objects for non-SDP relaxations:

1. Delta-style term partitions that identify relevant support terms by sign and exponent structure.
2. Newton-polytope geometry used to compute support vertices and barycentric combinations.

Together, these objects drive variable construction and inequality families in both geometric and SONC chapters.

4.7 Minimal Construction Pattern

```
from Irene.grouprings import CommutativeSemigroup, SemigroupAlgebra
from Irene.program import OptimizationProblem

S = CommutativeSemigroup(['x', 'y'])
SA = SemigroupAlgebra(S)
x = SA['x']
y = SA['y']

P = OptimizationProblem(SA)
P.set_objective(1 + x**2 + y**2)
P.add_constraints([1 - x**2, 1 - y**2])
```

CHAPTER

FIVE

SEMIDEFINITE PROGRAMMING

A *positive semidefinite* matrix is a symmetric real matrix whose eigenvalues are all nonnegative. A semidefinite programming problem is simply a linear program where the solutions are positive semidefinite matrices instead of points in Euclidean space.

5.1 Primal and Dual formulations

A typical semidefinite program (SDP for short) in the primal form is the following optimization problem:

$$\begin{cases} \min & \sum_{i=1}^{m} b_i x_i \\ \text{subject to} & \\ & \sum_{i=1}^{m} A_{ij} x_i - C_j \succeq 0 \quad j = 1, \ldots, k. \end{cases}$$

The dual program associated to the above SDP will be the following:

$$\begin{cases} \max & \sum_{j=1}^{k} tr(C_j \times Z_j) \\ \text{subject to} & \\ & \sum_{j=1}^{k} tr(A_{ij} \times Z_j) = b_i \quad i = 1, \ldots, m, \\ & Z_j \succeq 0 \qquad\qquad\qquad\quad j = 1, \ldots, k. \end{cases}$$

For convenience, we use a block representation for the matrices as follows:

$$C = \begin{pmatrix} C_1 & 0 & 0 & \ldots \\ 0 & C_2 & 0 & \ldots \\ \vdots & \ldots & \ddots & \vdots \\ 0 & \ldots & 0 & C_k \end{pmatrix},$$

and

$$A_i = \begin{pmatrix} A_{i1} & 0 & 0 & \ldots \\ 0 & A_{i2} & 0 & \ldots \\ \vdots & \ldots & \ddots & \vdots \\ 0 & \ldots & 0 & A_{ik} \end{pmatrix}.$$

This simplifies the k constraints of the primal form in to one constraint $\sum_{i=1}^{m} A_i x_i - C \succeq 0$ and the objective and constraints of the dual form as $tr(C \times Y)$ and $tr(A_i \times Z_i) = b_i$ for $i = 1, \ldots, m$.

5.2 The `sdp` class

The `sdp` class provides an interface to solve semidefinite programs using various range of well-known SDP solvers. Currently, the following solvers are supported:

5.2.1 CVXOPT

This is a python native convex optimization solver which can be obtained from CVXOPT. Beside semidefinite programs, it has various other solvers to handle convex optimization problems. In order to use this solver, the python package `CVXOPT` must be installed.

5.2.2 DSDP

If DSDP and `CVXOPT` are installed and `DSDP` is callable from command line, then it can be used as a SDP solver. Note that the current implementation uses `CVXOPT` to call `DSDP`, so `CVXOPT` is a requirement too.

5.2.3 SDPA

In case one manages to install SDPA and it can be called from command line, one can use `SDPA` as a SDP solver.

5.2.4 CSDP

Also, if csdp is installed and can be reached from command, then it can be used to solve SDP problems through `sdp` class.

To initialize and set the solver to one of the above simply use:

```
SDP = sdp('cvxopt') # initializes and uses `cvxopt` as solver.
```

Note

In windows, one can provide the path to each of the above solvers as the second parameter of the constructor:

```
SDP = sdp('csdp', {'csdp':"Path to executable csdp"}) # initializes and uses `csdp` as␣
↪solver existing at the given path.
```

5.2.5 Set the b vector:

To set the vector $b = (b_1, \ldots, b_m)$ one should use the method `sdp.SetObjective` which takes a list or a numpy array of numbers as b.

5.2.6 Set a block constraint:

To introduce the block of matrices $A_{i1}, \ldots, A_{ik}$ associated with x_i, one should use the method `sdp.AddConstraintBlock` that takes a list of matrices as blocks.

5.2.7 Set the constant block *C*:

The method `sdp.AddConstantBlock` takes a list of square matrices and use them to construct C.

5.2.8 Solve the input SDP:

To solve the input SDP simply call the method `sdp.solve()`. This will call the selected solver on the entered SDP and the output of the solver will be set as dictionary in `sdp.Info` with the following keys:

- `PObj`: The value of the primal objective.
- `DObj`: The value of the dual objective.
- `X`: The final X matrix.
- `Z`: The final Z matrix.

- `Status`: The final status of the solver.
- `CPU`: Total run time of the solver.

5.2.9 Example:

Consider the following SDP:

$$\begin{cases} \min & x_1 - x_2 + x_3 \\ \text{subject to} & \\ & \begin{pmatrix} 7 & 11 \\ 11 & -3 \end{pmatrix} x_1 + \begin{pmatrix} -7 & 18 \\ 18 & -8 \end{pmatrix} x_2 + \begin{pmatrix} 2 & 8 \\ 8 & -1 \end{pmatrix} x_3 \succeq \begin{pmatrix} -33 & 9 \\ 9 & -26 \end{pmatrix} \\ & \begin{pmatrix} 21 & 11 & 0 \\ 11 & -10 & -8 \\ 0 & -8 & -5 \end{pmatrix} x_1 + \begin{pmatrix} 0 & -10 & -16 \\ -10 & 10 & 10 \\ -16 & 10 & -3 \end{pmatrix} x_2 + \begin{pmatrix} 5 & -2 & 17 \\ -2 & 6 & -8 \\ 17 & -8 & -6 \end{pmatrix} x_3 \succeq \begin{pmatrix} -14 & -9 & -40 \\ -9 & -91 & -10 \\ -40 & -10 & -15 \end{pmatrix} \end{cases}$$

The following code solves the above program:

```
from numpy import matrix
from Irene import sdp
b = [1, -1, 1]
C = [matrix([[-33, 9], [9, -26]]),
     matrix([[-14, -9, -40], [-9, -91, -10], [-40, -10, -15]])]
A1 = [matrix([[7, 11], [11, -3]]),
      matrix([[21, 11, 0], [11, -10, -8], [0, -8, -5]])]
A2 = [matrix([[-7, 18], [18, -8]]),
      matrix([[0, -10, -16], [-10, 10, 10], [-16, 10, -3]])]
A3 = [matrix([[2, 8], [8, -1]]),
      matrix([[5, -2, 17], [-2, 6, -8], [17, -8, -6]])]
SDP = sdp('cvxopt')
SDP.SetObjective(b)
SDP.AddConstantBlock(C)
SDP.AddConstraintBlock(A1)
SDP.AddConstraintBlock(A2)
SDP.AddConstraintBlock(A3)
SDP.solve()
print SDP.Info
```

CHAPTER

SIX

OPTIMIZATION

Let X be a nonempty topological space and A be a unital sub-algebra of continuous functions over X which separates points of X. We consider the following optimization problem:

$$\begin{cases} \min & f(x) \\ \text{subject to} & \\ & g_i(x) \geq 0 \quad i = 1, \ldots, m. \end{cases}$$

Denote the feasibility set of the above program by K (i.e., $K = \{x \in X : g_i(x) \geq 0,\ i = 1, \ldots, m\}$). Let ρ be the optimum value of the above program and $\mathcal{M}_1^+(K)$ be the space of all probability Borel measures supported on K. One can show that:

$$\rho = \inf_{\mu \in \mathcal{M}_1^+(K)} \int f \, d\mu.$$

This associates a K-positive linear functional L_μ to every measure $\mu \in \mathcal{M}_1^+(K)$. Let us denote the set of all elements of A nonnegative on K by $Psd_A(K)$. If $\exists p \in Psd_A(K)$ such that $p^{-1}([0, n])$ is compact for each $n \in \mathbb{N}$, then one can show that every K-positive linear functional admits an integral representation via a Borel measure on K (Marshall's generalization of Haviland's theorem). Let $Q_{\mathbf{g}}$ be the quadratic module generated by $g_1, \ldots, g_m$, i.e, the set of all elements in A of the form

$$\sigma_0 + \sigma_1 g_1 + \cdots + \sigma_m g_m, \tag{6.1}$$

where $\sigma_0, \ldots, \sigma_m \in \sum A^2$ are sums of squares of elements of A. A quadratic module Q is said to be Archimedean if for every $h \in A$ there exists $M > 0$ such that $M \pm h \in Q$. By Jacobi's representation theorem, if Q is Archimedean and $h > 0$ on K, where $K = \{x \in X : g(x) \geq 0\ \forall g \in Q\}$, then $h \in Q$. Since Q is Archimedean, K is compact and this implies that if a linear functional on A is nonnegative on Q, then it is K-positive and hence admits an integral representation. Therefore:

$$\rho = \inf_{\substack{L(Q) \geq 0 \\ L(1) = 1}} L(f).$$

Let $Q = Q_{\mathbf{g}}$ and $L(Q) \subseteq [0, \infty)$. Then clearly $L(\sum A^2) \subseteq [0, \infty)$ which means L is positive semidefinite. Moreover, for each $i = 1, \ldots, m$, $L(g_i \sum A^2) \subseteq [0, \infty)$ which means the maps

$$\begin{array}{rccc} L_{g_i} : & A & \longrightarrow & \mathbb{R} \\ & h & \mapsto & L(g_i h) \end{array}$$

are positive semidefinite. So the optimum value of the following program is still equal to ρ:

$$\begin{cases} \min & L(f) \\ \text{subject to} & \\ & L \succeq 0 \\ & L_{g_i} \succeq 0 \quad i = 1, \ldots, m. \end{cases} \tag{6.2}$$

This is still not a semidefinite program, since each constraint is infinite dimensional. One plausible idea is to consider functionals on finite dimensional subspaces of A containing $f, g_1, \ldots, g_m$. This was done by Lasserre for polynomials [JBL].

Let $B \subseteq A$ be a linear subspace. If $L : A \longrightarrow \mathbb{R}$ is K-positive, so is its restriction on B. But generally, K-positive maps on B do not extend to K-positive one on A and hence existence of integral representations are not guaranteed. Under a rather mild condition, this issue can be resolved:

Theorem. [GIKM] Let $K \subseteq X$ be compact, $B \subseteq A$ a linear subspace such that there exists $p \in B$ strictly positive on K. Then every linear functional $L : B \longrightarrow \mathbb{R}$ satisfying $L(Psd_B(K)) \subseteq [0, \infty)$ admits an integral representation via a Borel measure supported on K.

Now taking B to be a finite dimensional linear space containing $f, g_1, \ldots, g_m$ and satisfying the assumptions of the above theorem, turns (6.2) into a semidefinite program. Note that this does not imply that the optimum value of the resulting SDP is equal to ρ since

- $Q_{\mathbf{g}} \cap B \neq Psd_B(K)$ and,
- there may not exist a decomposition of $f - \rho$ as in (6.1) inside B (i.e., the summands may not belong to B).

Thus, the optimum value gives only a lower bound for ρ. However, walking through a K-frame, as explained in [GIKM], constructs a net of lower bounds for ρ that eventually approaches ρ.

In practice, one needs a sufficiently large finite-dimensional linear space that contains $f, g_1, \ldots, g_m$ and admits a (6.1) decomposition of $f - \rho$ within that space. Therefore, convergence happens in finitely many steps, subject to finding a suitable K-frame for the problem.

The significance of this method is that it converts an optimization problem into finitely many semidefinite programs whose optimum values approach the optimum value of the original program, while each semidefinite program can be solved in polynomial time. This does not imply a polynomial-time algorithm for the full NP-complete optimization problem, because the number of SDP levels needed to reach the optimum is unknown, and no general bound is available in the Archimedean setting.

Note

1. One behavior that distinguishes this method from others is that using SDP relaxations always provides a lower bound for theminimum value of the objective function over the feasibility set. While other methods usually involve evaluation of the objectiveand hence the result is always an upper bound for the minimum.

2. The SDP relaxation method relies on symbolic computations which could be quite costly and slow. Therefore, dealing with rather large problems -although *Irene* takes advantage from multiple cores- can be rather slow.

6.1 Notation Bridge to Geometric and SONC Chapters

The same constrained problem data $(f, g_1, \ldots, g_m, K)$ is reused across all method families documented in this manual.

1. In the SDP hierarchy chapter, the central object is the moment functional L.
2. In the geometric chapter, transformed families h_j are built from f and g_i.
3. In the SONC chapter, the central transformed expression is $G(\mu) = f - \sum_i \mu_i g_i$.

This shared notation is intentional and helps compare lower-bound certificates across SDP, GP, and SONC formulations on the same POP instance.

6.2 Beyond SDP: Alternative Lower-Bound Mechanisms

The remainder of this documentation extends the same POP template to two additional lower-bound mechanisms:

1. **Geometric programming relaxations, which use transformed polynomial families**
   and sign/support constraints to produce GP certificates.
2. **SONC relaxations, which use circuit-structured support geometry and**
   barycentric weights.

These alternatives do not replace the SDP hierarchy. Instead, they provide complementary certificates that can be more suitable for specific sparse, high-degree, or structure-rich instances.

6.2.1 Polynomial Optimization

The SDP relaxation method was originally introduced by Lasserre [JBL] for polynomial optimization problems and excellent software packages such as GloptiPoly and ncpol2sdpa exist to handle constraint polynomial optimization problems.

Irene uses sympy for symbolic computations, so it always needs to be imported and the symbolic variables must be introduced. Once these steps are done, the objective and constraints should be entered using `SetObjective` and *AddConstraint* methods. The method `MomentsOrd` takes the relaxation degree upon user's request, otherwise the minimum relaxation degree will be used. The default SDP solver is `CVXOPT` which can be modified via `SetSDPSolver` method. Currently `CVXOPT`, `DSDP`, `SDPA` and `CSDP` are supported. The next step is initialization of the SDP by `InitSDP` and finally solving the SDP via `Minimize`. The output is stored in the `Solution` variable as a Python dictionary.

Example Solve the following polynomial optimization problem:

$$\begin{cases} \min & -2x + y - z \\ \text{subject to} & 24 - 20x + 9y - 13z + 4x^2 - 4xy \\ & +4xz + 2y^2 - 2yz + 2z^2 \geq 0 \\ & x + y + z \leq 4 \\ & 3y + z \leq 6 \\ & 0 \leq x \leq 2 \\ & y \geq 0 \\ & 0 \leq z \leq 3. \end{cases}$$

The following program uses relaxation of degree 3 and *sdpa* to solve the above problem:

```
from sympy import *
from Irene import *
# introduce variables
x = Symbol('x')
y = Symbol('y')
z = Symbol('z')
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y, z])
# set the objective
Rlx.SetObjective(-2 * x + y - z)
# add support constraints
Rlx.AddConstraint(24 - 20 * x + 9 * y - 13 * z + 4 * x**2 -
                  4 * x * y + 4 * x * z + 2 * y**2 - 2 * y * z + 2 * z**2 >= 0)
Rlx.AddConstraint(x + y + z <= 4)
Rlx.AddConstraint(3 * y + z <= 6)
Rlx.AddConstraint(x >= 0)
Rlx.AddConstraint(x <= 2)
Rlx.AddConstraint(y >= 0)
```

(continues on next page)

(continued from previous page)

```
Rlx.AddConstraint(z >= 0)
Rlx.AddConstraint(z <= 3)
# set the relaxation order
Rlx.MomentsOrd(3)
# set the solver
Rlx.SetSDPSolver('dsdp')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
# output
print Rlx.Solution
```

The output looks like:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 8.04711222649 seconds
              Run Time: 1.056733 seconds
Primal Objective Value: -4.06848294478
  Dual Objective Value: -4.06848289445
Feasible solution for moments of order 3
```

6.3 Moment Constraints

Initially the only constraints forced on the moments are those in (6.2). We can also force user defined constraints on the moments by calling `MomentConstraint` on a `Mom` object. The following adds two constraints $\int xy\, d\mu \geq \frac{1}{2}$ and $\int yz\, d\mu + \int z\, d\mu \geq 1$ to the previous example:

```
from sympy import *
from Irene import *
# introduce variables
x = Symbol('x')
y = Symbol('y')
z = Symbol('z')
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y, z])
# set the objective
Rlx.SetObjective(-2 * x + y - z)
# add support constraints
Rlx.AddConstraint(24 - 20 * x + 9 * y - 13 * z + 4 * x**2 -
                  4 * x * y + 4 * x * z + 2 * y**2 - 2 * y * z + 2 * z**2 >= 0)
Rlx.AddConstraint(x + y + z <= 4)
Rlx.AddConstraint(3 * y + z <= 6)
Rlx.AddConstraint(x >= 0)
Rlx.AddConstraint(x <= 2)
Rlx.AddConstraint(y >= 0)
Rlx.AddConstraint(z >= 0)
Rlx.AddConstraint(z <= 3)
# add moment constraints
```

(continues on next page)

(continued from previous page)

```
Rlx.MomentConstraint(Mom(x * y) >= .5)
Rlx.MomentConstraint(Mom(y * z) + Mom(z) >= 1)
# set the relaxation order
Rlx.MomentsOrd(3)
# set the solver
Rlx.SetSDPSolver('dsdp')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
# output
print Rlx.Solution
print "Moment of x*y:", Rlx.Solution[x * y]
print "Moment of y*z + z:", Rlx.Solution[y * z] + Rlx.Solution[z]
```

Solution is:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 7.91646790504 seconds
              Run Time: 1.041935 seconds
Primal Objective Value: -4.03644346623
  Dual Objective Value: -4.03644340796
Feasible solution for moments of order 3

Moment of x*y: 0.500000001712
Moment of y*z + z: 2.72623169152
```

6.4 Equality Constraints

Although it is possible to add equality constraints via `AddConstraint` and `MomentConstraint`, but *SDPRelaxation* converts them to two inequalities and considers a certain margin of error. For $A = B$, it considers $A \geq B - \varepsilon$ and $A \leq B + \varepsilon$. In this case the value of ε can be modified by setting *SDPRelaxation.ErrorTolerance* which its default value is 10^{-6}.

6.4.1 Truncated Moment Problem

It must be clear that we can use `SDPRelaxations.MomentConstraint` to introduce a typical truncated moment problem over polynomials as described in [JNie].

Example Find the support of a measure μ whose support is a subset of $[-1, 1]^2$ and the followings hold:

$$\begin{array}{cc} \int x^2 d\mu = \int y^2 d\mu = \frac{1}{3} & \int x^2 y d\mu = \int x y^2 d\mu = 0 \\ \int x^2 y^2 d\mu = \frac{1}{9} & \int x^4 y^2 d\mu = \int x^2 y^4 d\mu = \frac{1}{15}. \end{array}$$

The following code does the job:

```
from sympy import *
from Irene import *
# introduce variables
x = Symbol('x')
```

(continues on next page)

(continued from previous page)

```
y = Symbol('y')
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y])
# add support constraints
Rlx.AddConstraint(1 - x**2 >= 0)
Rlx.AddConstraint(1 - y**2 >= 0)
# add moment constraints
Rlx.MomentConstraint(Mom(x**2) == 1. / 3.)
Rlx.MomentConstraint(Mom(y**2) == 1. / 3.)
Rlx.MomentConstraint(Mom(x**2 * y) == 0.)
Rlx.MomentConstraint(Mom(x * y**2) == 0.)
Rlx.MomentConstraint(Mom(x**2 * y**2) == 1. / 9.)
Rlx.MomentConstraint(Mom(x**4 * y**2) == 1. / 15.)
Rlx.MomentConstraint(Mom(x**2 * y**4) == 1. / 15.)
# set the solver
Rlx.SetSDPSolver('dsdp')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
# output
Rlx.Solution.ExtractSolution('lh', 2)
print Rlx.Solution
```

and the result is:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 1.08686900139 seconds
              Run Time: 0.122459 seconds
Primal Objective Value: 0.0
  Dual Objective Value: -9.36054051771e-09
               Support:
                (0.40181215311129925, 0.54947643681480196)
                (-0.40181215311127805, -0.54947643681498193)
        Support solver: Lasserre--Henrion
Feasible solution for moments of order 3
```

Note that the solution is not necessarily unique.

6.4.2 Optimization of Rational Functions

Given two polynomials $p(X), q(X), g_1(X), \ldots, g_m(X)$, the minimum of $\frac{p(X)}{q(X)}$ over $K = \{x : g_i(x) \geq 0,\ i = 1, \ldots, m\}$ is equal to

$$\begin{cases} \min & \int p(X)\, d\mu \\ \text{subject to} & \\ & \int q(X)\, d\mu = 1, \\ & \mu \in \mathcal{M}^+(K). \end{cases}$$

Note that in this case μ is not taken to be a probability measure, but instead $\int q(X)\, d\mu = 1$. We can use `SDPRelaxations.Probability = False` to relax the probability condition on μ and use moment constraints to enforce $\int q(X)\, d\mu = 1$. The following example explains this.

Example: Find the minimum of $\frac{x^2-2x}{x^2+2x+1}$:

```
from sympy import *
from Irene import *
# define the symbolic variable
x = Symbol('x')
# initiate the SDPRelaxations object
Rlx = SDPRelaxations([x])
# settings
Rlx.Probability = False
# set the objective
Rlx.SetObjective(x**2 - 2*x)
# moment constraint
Rlx.MomentConstraint(Mom(x**2+2*x+1) == 1)
# set the sdp solver
Rlx.SetSDPSolver('cvxopt')
# initiate the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
```

The result is:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 0.167912006378 seconds
              Run Time: 0.008987 seconds
Primal Objective Value: -0.333333666913
  Dual Objective Value: -0.333333667469
Feasible solution for moments of order 1
```

Note

Beside `SDPRelaxations.Probability` there is another attribute `SDPRelaxations.PSDMoment` which by default is set to `True` and makes sure that the sdp solver assumes positivity for the moment matrix.

6.4.3 Optimization over Varieties

Now we employ the results of [GIKM] to solve more complex optimization problems. The main idea is to represent the given function space as a quotient of a suitable polynomial algebra.

Suppose that we want to optimize the function $\sqrt[3]{(xy)^2} - x + y^2$ over the closed disk with radius 3. In order to deal with the term $\sqrt[3]{(xy)^2}$, we introduce an algebraic relation to `SDPRelaxations` object and give a monomial order for Groebner basis computations (default is *lex* for lexicographic order). Clearly $xy - \sqrt[3]{(xy)}^3 = 0$. Therefore by introducing an auxiliary variable or function symbol, say $f(x, y)$ the problem can be stated in the quotient of $\frac{\mathbb{R}[x,y,f]}{\langle xy-f^3\rangle}$. To check the result of `SDPRelaxations` we employ `scipy.optimize.minimize` with two solvers `COBYLA` and `COBYLA` as well as two solvers, *Augmented Lagrangian Particle Swarm Optimizer* and *Non Sorting Genetic Algorithm II* from pyOpt:

```
from sympy import *
from Irene import *
# introduce variables
x = Symbol('x')
y = Symbol('y')
f = Function('f')(x, y)
# define algebraic relations
rel = [x * y - f**3]
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y, f], rel)
# set the monomial order
Rlx.SetMonoOrd('lex')
# set the objective
Rlx.SetObjective(f**2 - x + y**2)
# add support constraints
Rlx.AddConstraint(9 - x**2 - y**2 >= 0)
# set the solver
Rlx.SetSDPSolver('cvxopt')
# Rlx.MomentsOrd(2)
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
# output
print Rlx.Solution
# using scipy
from numpy import power
from scipy.optimize import minimize
fun = lambda x: power(x[0]**2 * x[1]**2, 1. / 3.) - x[0] + x[1]**2
cons = (
    {'type': 'ineq', 'fun': lambda x: 9 - x[0]**2 - x[1]**2})
sol1 = minimize(fun, (0, 0), method='COBYLA', constraints=cons)
sol2 = minimize(fun, (0, 0), method='SLSQP', constraints=cons)
print "solution according to 'COBYLA'"
print sol1
print "solution according to 'SLSQP'"
print sol2

# pyOpt
from pyOpt import *

def objfunc(x):
        from numpy import power
        f = power(x[0]**2 * x[1]**2, 1. / 3.) - x[0] + x[1]**2
        g = [x[0]**2 + x[1]**2 - 9]
        fail = 0
        return f, g, fail

opt_prob = Optimization('A third root function', objfunc)
opt_prob.addVar('x1', 'c', lower=-3, upper=3, value=0.0)
opt_prob.addVar('x2', 'c', lower=-3, upper=3, value=0.0)
opt_prob.addObj('f')
opt_prob.addCon('g1', 'i')
```

(continues on next page)

(continued from previous page)

```
# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

The output will be:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 0.12473487854 seconds
              Run Time: 0.004865 seconds
Primal Objective Value: -2.99999997394
  Dual Objective Value: -2.9999999473
Feasible solution for moments of order 1

solution according to 'COBYLA'
     fun: -0.99788411120450926
   maxcv: 0.0
 message: 'Optimization terminated successfully.'
    nfev: 25
  status: 1
 success: True
       x: array([  9.99969494e-01,   9.52333693e-05])
 solution according to 'SLSQP'
     fun: -2.9999975825413681
     jac: array([  -0.99999923,  689.00398242,    0.        ])
 message: 'Optimization terminated successfully.'
    nfev: 64
     nit: 13
    njev: 13
  status: 0
 success: True
       x: array([  3.00000000e+00,  -1.25290367e-09])

ALPSO Solution to A third root function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.1174
    Total Function Evaluations:      1720
    Lambda: [ 0.00023458]
    Seed: 1482111093.38230896

    Objectives:
```

(continues on next page)

(continued from previous page)

```
        Name        Value        Optimum
             f        -2.99915             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       3.000000      -3.00e+00     3.00e+00
             x2       c       0.000008      -3.00e+00     3.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to A third root function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.3833
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
             f        -2.99898             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       3.000000      -3.00e+00     3.00e+00
             x2       c      -0.000011      -3.00e+00     3.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= -0.000000 <= 0.00e+00

--------------------------------------------------------------------------------
```

6.4.4 Optimization over arbitrary functions

Any given algebra can be represented as a quotient of a suitable polynomial algebra (on possibly infinitely many variables). Since optimization problems usually involve finitely many functions and constraints, we can apply the technique introduced in the previous section, as soon as we figure out the quotient representation of the function space.

Let us walk through the procedure by solving some examples.

Example 1. Find the optimum value of the following program:

$$\begin{cases} \min & -(\sin(x)-1)^3-(\sin(x)-\cos(y))^4-(\cos(y)-3)^2 \\ \text{subject to} & \\ & 10-(\sin(x)-1)^2 \geq 0, \\ & 10-(\sin(x)-\cos(y))^2 \geq 0, \\ & 10-(\cos(y)-3)^2 \geq 0. \end{cases}$$

Let us introduce four symbols to represent trigonometric functions:

$f: \quad \sin(x)$	$g: \quad \cos(x)$
$h: \quad \sin(y)$	$k: \quad \cos(y)$

Then the quotient algebra $\frac{\mathbb{R}[f,g,h,k]}{I}$ where $I = \langle f^2+g^2-1, h^2+k^2-1\rangle$ is the right framework to solve the optimization problem. We also compare the outcome of `SDPRelaxations` with `scipy` and `pyswarm`:

```
from sympy import *
from Irene import *
# introduce variables
x = Symbol('x')
f = Function('f')(x)
g = Function('g')(x)
h = Function('h')(x)
k = Function('k')(x)
# define algebraic relations
rels = [f**2 + g**2 - 1, h**2 + k**2 - 1]
# initiate the Relaxation object
Rlx = SDPRelaxations([f, g, h, k], rels)
# set the monomial order
Rlx.SetMonoOrd('lex')
# set the objective
Rlx.SetObjective(-(f - 1)**3 - (f - k)**4 - (k - 3)**2)
# add support constraints
Rlx.AddConstraint(10 - (f - 1)**2 >= 0)
Rlx.AddConstraint(10 - (f - k)**2 >= 0)
Rlx.AddConstraint(10 - (k - 3)**2 >= 0)
# set the solver
Rlx.SetSDPSolver('csdp')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
# output
print Rlx.Solution
# using scipy
from scipy.optimize import minimize
fun = lambda x: -(sin(x[0]) - 1)**3 - (sin(x[0]) -
                                        cos(x[1]))**4 - (cos(x[1]) - 3)**2
cons = (
    {'type': 'ineq', 'fun': lambda x: 10 - (sin(x[0]) - 1)**2},
    {'type': 'ineq', 'fun': lambda x: 10 - (sin(x[0]) - cos(x[1]))**2},
    {'type': 'ineq', 'fun': lambda x: 10 - (cos(x[1]) - 3)**2})
sol1 = minimize(fun, (0, 0), method='COBYLA', constraints=cons)
sol2 = minimize(fun, (0, 0), method='SLSQP', constraints=cons)
```

(continues on next page)

(continued from previous page)

```
print "solution according to 'COBYLA':"
print sol1
print "solution according to 'SLSQP':"
print sol2
# pyOpt
from pyOpt import *


def objfunc(x):
    from numpy import sin, cos
    f = -(sin(x[0]) - 1)**3 - (sin(x[0]) - cos(x[1]))**4 - (cos(x[1]) - 3)**2
    g = [
        (sin(x[0]) - 1)**2 - 10,
        (sin(x[0]) - cos(x[1]))**2 - 10,
        (cos(x[1]) - 3)**2 - 10
    ]
    fail = 0
    return f, g, fail

opt_prob = Optimization('A trigonometric function', objfunc)
opt_prob.addVar('x1', 'c', lower=-10, upper=10, value=0.0)
opt_prob.addVar('x2', 'c', lower=-10, upper=10, value=0.0)
opt_prob.addObj('f')
opt_prob.addCon('g1', 'i')
opt_prob.addCon('g2', 'i')
opt_prob.addCon('g3', 'i')
# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

Solutions are:

```
Solution of a Semidefinite Program:
                Solver: CSDP
                Status: Optimal
   Initialization Time: 3.22915506363 seconds
              Run Time: 0.016662 seconds
Primal Objective Value: -12.0
  Dual Objective Value: -12.0
Feasible solution for moments of order 2

solution according to 'COBYLA':
     fun: -11.824901993777621
   maxcv: 1.7763568394002505e-15
 message: 'Optimization terminated successfully.'
    nfev: 42
  status: 1
```

(continues on next page)

(continued from previous page)

```
 success: True
       x: array([ 1.57064986,  1.7337948 ])
solution according to 'SLSQP':
     fun: -11.9999999999720
     jac: array([ -2.94446945e-05,  -1.78813934e-05,   0.00000000e+00])
 message: 'Optimization terminated successfully.'
    nfev: 23
     nit: 5
    njev: 5
  status: 0
 success: True
       x: array([ -1.57079782e+00,  -6.42618794e-07])

ALPSO Solution to A trigonometric function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.3503
    Total Function Evaluations:      3640
    Lambda: [ 0.         0.         2.0077542]
    Seed: 1482111691.32805490

    Objectives:
        Name        Value        Optimum
             f        -11.8237             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       7.854321      -1.00e+01     1.00e+01
             x2       c       4.549489      -1.00e+01     1.00e+01

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= -10.000000 <= 0.00e+00
             g2           i       -1.00e+21 <= -8.649336 <= 0.00e+00
             g3           i       -1.00e+21 <= -0.000612 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to A trigonometric function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.7216
    Total Function Evaluations:
```

(continues on next page)

(continued from previous page)

```
    Objectives:
        Name        Value        Optimum
             f             -12             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -7.854036      -1.00e+01     1.00e+01
             x2       c       0.000004      -1.00e+01     1.00e+01

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= -6.000000 <= 0.00e+00
             g2           i       -1.00e+21 <= -6.000000 <= 0.00e+00
             g3           i       -1.00e+21 <= -6.000000 <= 0.00e+00

--------------------------------------------------------------------------------
```

6.4.5 SOS Decomposition

Let f_* be the result of `SDPRelaxations.Minimize()`, then $f - f_* \in Q_{\mathbf{g}}$. Therefore, there exist $\sigma_0, \sigma_1, \ldots, \sigma_m \in \sum A^2$ such that $f - f_* = \sigma_0 + \sum_{i=1}^m \sigma_i g_i$. Once the `Minimize()` is called, the method `SDPRelaxations.Decompose()` returns this a dictionary of elements of A of the form `{0:[a(0, 1), ..., a(0, k_0)], ..., m:[a(m, 1), ..., a(m, k_m)}` such that

$$f - f_* = \sum_{i=0}^{m} g_i \sum_{j=1}^{k_i} a_{ij}^2,$$

where $g_0 = 1$.

Usually there are extra coefficients that are very small in absolute value as a result of round off error that should be ignored.

The following example shows how to employ this functionality:

```
from sympy import *
from Irene import SDPRelaxations
# define the symbolic variables and functions
x = Symbol('x')
y = Symbol('y')
z = Symbol('z')

Rlx = SDPRelaxations([x, y, z])
Rlx.SetObjective(x**3 + x**2 * y**2 + z**2 * x * y - x * z)
Rlx.AddConstraint(9 - (x**2 + y**2 + z**2) >= 0)
# initiate the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
# extract decomposition
V = Rlx.Decompose()
# test the decomposition
```

(continues on next page)

(continued from previous page)

```
sos = 0
for v in V:
    # for g0 = 1
    if v == 0:
        sos = expand(Rlx.ReduceExp(sum([p**2 for p in V[v]])))
    # for g1, the constraint
    else:
        sos = expand(Rlx.ReduceExp(
            sos + Rlx.Constraints[v - 1] * sum([p**2 for p in V[v]])))
sos = sos.subs(Rlx.RevSymDict)
pln = Poly(sos).as_dict()
pln = {ex:round(pln[ex],5) for ex in pln}
print Poly(pln, (x,y,z)).as_expr()
```

The output looks like this:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 0.875229120255 seconds
              Run Time: 0.031426 seconds
Primal Objective Value: -27.4974076889
  Dual Objective Value: -27.4974076213
Feasible solution for moments of order 2

1.0*x**3 + 1.0*x**2*y**2 + 1.0*x*y*z**2 - 1.0*x*z + 27.49741
```

6.4.6 The `Resume` method

It happens from time to time that one needs to stop the process of `SDPRelaxations` to look into its progress and/or run the code later. This has been accommodated thanks to python's support for serialization and error handling. Since the initialization of the final SDP is the most time consuming part of the process, if one breaks this via *Ctrl-c*, the object will save all the computation that has been done so far in a *.rlx* file named with the name of the object. So, if one wants to resume the process later, it suffices to call the `Resume` method after instantiation and leave the program out and continue the initialization via calling `InitSDP` method.

6.4.7 The `SDRelaxSol`

This object is a container for the solution of `SDPRelaxation` objects. It contains the following informations:

- *Primal*: the value of the SDP in primal form,
- *Dual*: the value of the SDP in dual form,
- *RunTime*: the run time of the sdp solver,
- *InitTime*: the total time consumed for initialization of the sdp,
- *Solver*: the name of sdp solver,
- *Status*: final status of the sdp solver,
- *RelaxationOrd*: order of relaxation,
- *TruncatedMmntSeq*: a dictionary of resulted moments,
- *MomentMatrix*: the resulted moment matrix,

- *ScipySolver*: the scipy solver to extract solutions,
- *err_tol*: the minimum value which is considered to be nonzero,
- *Support*: the support of discrete measure resulted from `SDPRelaxation.Minimize()`,
- *Weights*: corresponding weights for the Dirac measures.

The `SDRelaxSol` after initiation is an iterable object. The moments can be retrieved by passing the index to the iterable `SDRelaxSol[idx]`.

6.5 Extracting solutions

By default, the support of the measure is not calculated, but it can be approximated by calling the method `SDRelaxSol.ExtractSolution()`.

There exists an exact theoretical method for extracting the support of the solution measure as explained in [HL]. But because of the numerical error of sdp solvers, computing rank and hence the support is quite difficult. So, `SDRelaxSol.ExtractSolution()` estimates the rank numerically by assuming that eigenvalues with absolute value less than `err_tol` which by default is set to `SDPRelaxation.ErrorTolerance`.

Two methods are implemented for extracting solutions:

- **Lasserre-Henrion** method as explained in [HL]. To employ this method simply call `SDRelaxSol.ExtractSolution('LH', card)`, where `card` is the maximum cardinality of the support.
- **Moment Matching** method which employs `scipy.optimize.root` to approximate the support. The default `scipy` solver is set to *lm*, but other solvers can be selected using `SDRelaxSol.SetScipySolver(solver)`. It is not guaranteed that scipy solvers return a reliable answer, but modifying sdp solvers and other parameters like `SDPRelaxation.ErrorTolerance` may help to get better results. To use this method call `SDRelaxSol.ExtractSolution('scipy', card)` where `card` is as above.

Example 1. Solve and find minimizers of $x^2 + y^2 + z^4$ where $x + y + z = 4$:

```
from sympy import *
from Irene import *

x, y, z = symbols('x,y,z')

Rlx = SDPRelaxations([x, y, z])
Rlx.SetSDPSolver('cvxopt')
Rlx.SetObjective(x**2 + y**2 + z**4)
Rlx.AddConstraint(Eq(x + y + z, 4))
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
# extract support
Rlx.Solution.ExtractSolution('LH', 1)
print Rlx.Solution

# pyOpt
from pyOpt import *

def objfunc(x):
        f = x[0]**2 + x[1]**2 + x[2]**4
        g = [x[0] + x[1] + x[2] - 4]
        fail = 0
```

(continues on next page)

(continued from previous page)

```
        return f, g, fail

opt_prob = Optimization('Testing solutions', objfunc)
opt_prob.addVar('x1', 'c', lower=-4, upper=4, value=0.0)
opt_prob.addVar('x2', 'c', lower=-4, upper=4, value=0.0)
opt_prob.addVar('x3', 'c', lower=-4, upper=4, value=0.0)
opt_prob.addObj('f')
opt_prob.addCon('g1', 'e')
# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
```

The output is:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 1.59334087372 seconds
              Run Time: 0.021102 seconds
Primal Objective Value: 5.45953579912
  Dual Objective Value: 5.45953586121
               Support:
                (0.91685039306810523, 1.541574317520042, 1.5415743175200163)
        Support solver: Lasserre--Henrion
Feasible solution for moments of order 2

ALPSO Solution to Testing solutions
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.1443
    Total Function Evaluations:      1720
    Lambda: [-3.09182651]
    Seed: 1482274189.55335808

    Objectives:
        Name        Value        Optimum
             f         5.46051             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       1.542371      -4.00e+00     4.00e+00
             x2       c       1.541094      -4.00e+00     4.00e+00
             x3       c       0.916848      -4.00e+00     4.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1       e                0.000314 = 0.00e+00
```

(continues on next page)

(continued from previous page)

```
--------------------------------------------------------------------------------
```

Example 2. Minimize $-(x-1)^2-(x-y)^2-(y-3)^2$ where $1-(x-1)^2 \geq 0$, $1-(x-y)^2 \geq 0$ and $1-(y-3)^2 \geq 0$. It has three minimizers $(2,3)$, $(1,2)$, and $(2,2)$:

```
from sympy import *
from Irene import *

x, y = symbols('x, y')

Rlx = SDPRelaxations([x, y])
Rlx.SetSDPSolver('csdp')
Rlx.SetObjective(-(x - 1)**2 - (x - y)**2 - (y - 3)**2)
Rlx.AddConstraint(1 - (x - 1)**2 >= 0)
Rlx.AddConstraint(1 - (x - y)**2 >= 0)
Rlx.AddConstraint(1 - (y - 3)**2 >= 0)
Rlx.MomentsOrd(2)
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
# extract support
Rlx.Solution.ExtractSolution('LH')
print Rlx.Solution

# pyOpt
from pyOpt import *


def objfunc(x):
    f = -(x[0] - 1)**2 - (x[0] - x[1])**2 - (x[1] - 3)**2
    g = [
        (x[0] - 1)**2 - 1,
        (x[0] - x[1])**2 - 1,
        (x[1] - 3)**2 - 1
    ]
    fail = 0
    return f, g, fail

opt_prob = Optimization("Lasserre's Example", objfunc)
opt_prob.addVar('x1', 'c', lower=-3, upper=3, value=0.0)
opt_prob.addVar('x2', 'c', lower=-3, upper=3, value=0.0)
opt_prob.addObj('f')
opt_prob.addCon('g1', 'i')
opt_prob.addCon('g2', 'i')
opt_prob.addCon('g3', 'i')
# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
```

(continues on next page)

(continued from previous page)

```
nsg2(opt_prob)
print opt_prob.solution(1)
```

which results in:

```
Solution of a Semidefinite Program:
                Solver: CSDP
                Status: Optimal
   Initialization Time: 0.861004114151 seconds
              Run Time: 0.00645 seconds
Primal Objective Value: -2.0
  Dual Objective Value: -2.0
               Support:
                (2.000000006497352, 3.000000045123556)
                (0.99999993829586131, 1.9999999487412694)
                (1.9999999970209055, 1.9999999029899564)
        Support solver: Lasserre--Henrion
Feasible solution for moments of order 2


ALPSO Solution to Lasserre's Example
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.1353
    Total Function Evaluations:      1720
    Lambda: [ 0.08278879  0.08220848  0.        ]
    Seed: 1482307696.27431393

    Objectives:
        Name        Value        Optimum
             f              -2             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       1.999967      -3.00e+00     3.00e+00
             x2       c       3.000000      -3.00e+00     3.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= -0.000065 <= 0.00e+00
             g2           i       -1.00e+21 <= 0.000065 <= 0.00e+00
             g3           i       -1.00e+21 <= -1.000000 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to Lasserre's Example
================================================================================
```

(continues on next page)

(continued from previous page)

```
        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.2406
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
             f        -1.99941             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       1.999947      -3.00e+00     3.00e+00
             x2       c       2.000243      -3.00e+00     3.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= -0.000106 <= 0.00e+00
             g2           i       -1.00e+21 <= -1.000000 <= 0.00e+00
             g3           i       -1.00e+21 <= -0.000486 <= 0.00e+00

--------------------------------------------------------------------------------
```

Irene detects all minimizers correctly, but each *pyOpt* solvers only detect one. Note that we did not specify number of solutions, but the solver extracted them all.

CHAPTER

SEVEN

GEOMETRIC PROGRAMMING RELAXATIONS

The module `geometric.py` implements a geometric-programming lower-bound strategy for constrained polynomial optimization over basic semialgebraic sets.

7.1 Theory to Implementation

The implementation follows the Section 4 construction used in the repository notes, including a transformed program defined by a matrix H and a GP objective/constraint system over auxiliary variables.

Let $g_0 = -f$ and let $g_1, \ldots, g_m$ be the constraint polynomials. The geometric pipeline builds transformed expressions h_j using H and then solves for a lower bound through GP variables μ, w_α, and z_α.

At a high level, the transformed family is:

$$h_j = \sum_k H_{k,j} g_k.$$

The relaxation objective then combines transformed positive parts and residual terms indexed by active support elements. In implementation-oriented notation, the objective has the form

$$p(\mu, w, z) = \sum_{j=1}^{m} h_+(h_j)\mu_j + \sum_{\alpha \in \Delta^{<d}} (d - \deg \alpha) \left(\frac{w_\alpha}{d}\right)^{\frac{d}{d-\deg \alpha}} \prod_i \left(\frac{\alpha_i}{z_{\alpha,i}}\right)^{\frac{\alpha_i}{d-\deg \alpha}}.$$

Here d denotes the effective even relaxation order used by the problem.

The implementation follows the lower-bound construction associated with Section 4, including equation (3) as implemented in `geometric.py` for transformed objective and constraint families.

The main implementation stages are:

1. `auto_transform_matrix`: builds a transformation matrix satisfying the required sign structure.
2. `transform_program`: constructs transformed expressions from the original objective/constraints.
3. `_build_delta_sets` and `_initialize_variables`: prepare monomial sets and GP variables.
4. `_build_objective` and constraint assembly in `solve`: formulate and solve the GP model.

7.2 Interpretation of the Matrix Transform

The matrix H is not only a numerical preconditioner. It defines which linear combinations of objective/constraint polynomials are exposed to the GP model and therefore determines the sign and support structure of the final inequalities. The `auto_transform_matrix` routine computes a feasible transformation using diagonal/sign conditions encoded in the implementation.

7.3 Constraint Families in `solve`

The method assembles several GP/signomial constraint blocks:

1. Normalization and variable bounds (including $\mu_0 = 1$).
2. Generator-wise inequalities over transformed coefficients.
3. Degree-matching constraints for terms in $\Delta^{=d}$.
4. Positive/negative coefficient balancing for all active terms.

These blocks correspond to the computational implementation of the Section 4 equation family and are solved through `gpkit.Model`.

7.4 Lower-Bound Meaning

The solved value is used as a lower bound certificate for the original POP. As with any hierarchy-style relaxation strategy, bound quality depends on model order, transform quality, and problem structure.

7.5 Practical Notes

1. The method returns a lower bound as a floating-point value.
2. Solver availability and numerical conditioning can affect runtime behavior.
3. `examples/GPExample.py` provides a complete end-to-end usage pattern.
4. If automatic transformation is unstable for a given instance, a custom matrix can be supplied by setting `gp.H` before calling `solve`.

CHAPTER

EIGHT

SONC RELAXATIONS

The module `sonc.py` implements constrained SONC relaxations as geometric programs.

8.1 Formulation Focus

The implementation follows the Section 3 constrained SONC structure used in this project and supports the Example 3.3 workflow in the repository examples.

The central transformed object is:

$$G(\mu) = f - \sum_i \mu_i g_i.$$

For each relevant exponent term, the model introduces auxiliary variables and constraints that bound positive and negative parts of coefficients and connect them through barycentric weights.

For each β in the active delta set and each support point $\alpha(j)$, the implementation uses the standard constrained SONC pattern with variables $a_{\beta,j}$, b_β, and multipliers μ. To align with the Section 3 constrained formulation (equation (3.3)), the key families implemented in code are:

8.2 Variable Roles

1. μ: Lagrange-style multipliers combining objective and constraints through $G(\mu)$.
2. $a_{\beta,j}$: circuit weights attached to support points for each active β.
3. b_β: auxiliary upper bounds controlling positive and negative parts of $G(\mu)_\beta$.

(F1) support-point inequalities

$$\sum_j a_{\beta,j} \leq G(\mu)_{\alpha(j)},$$

(F2) circuit-product inequality

$$\prod_j \left(\frac{a_{\beta,j}}{\lambda_j^{(\beta)}} \right)^{\lambda_j^{(\beta)}} \geq b_\beta,$$

(F3) positive-part bound

$$G(\mu)_\beta^+ \leq b_\beta,$$

(F4) negative-part bound

$$G(\mu)_\beta^- \leq b_\beta.$$

The GP objective minimized in this chapter is denoted by $p(\mu, a, b)$, consistent with the Section 3 notation used by the constrained SONC formulation.

In implementation-oriented form, the objective can be read as:

$$p(\mu, a, b) = \sum_{i=1}^{m} \mu_i g_i^+(\alpha_0) + \sum_{\beta:\lambda_0^{(\beta)}>0} \lambda_0^{(\beta)} b_\beta^{1/\lambda_0^{(\beta)}} \prod_{j\neq 0} \left(\frac{\lambda_j^{(\beta)}}{a_{\beta,j}} \right)^{\lambda_j^{(\beta)}/\lambda_0^{(\beta)}} .$$

This is the objective assembled in `_build_objective` after barycentric data is available.

The barycentric weights $\lambda^{(\beta)}$ are computed from support points via convex-combination routines before model assembly.

8.3 Branching on $\lambda_0^{(\beta)}$

The implementation distinguishes two geometric cases:

1. $\lambda_0^{(\beta)} > 0$: objective includes a weighted exponential/product term for β.
2. $\lambda_0^{(\beta)} = 0$: circuit-product inequality (F2) controls b_β directly without the λ_0 objective branch.

This split matches the constrained SONC structure implemented in the solver assembly pipeline.

8.4 Implementation Stages in `SONCRelaxations`

1. `_build_delta_sets`: collects candidate terms.
2. `_build_support_points`: computes support points/Newton vertices.
3. `_build_beta_info`: computes barycentric coordinates.
4. `_initialize_variables`: allocates variables for multipliers and circuit terms.
5. `_build_constraints` and `_build_objective`: constructs the constrained GP.
6. `solve`: solves and returns a lower bound.

8.5 Theory-to-Code Anchors

1. `_build_delta_sets` corresponds to active β extraction.
2. `_build_support_points` and `_build_beta_info` provide support vertices and barycentric weights $\lambda^{(\beta)}$.
3. `_g_split` computes positive and negative parts of coefficients in $G(\mu)$.
4. `_build_constraints` encodes the constrained SONC families.
5. `_build_objective` builds the GP objective over μ, a, and b terms.

8.6 Geometric Interpretation

At a high level, SONC uses support geometry to construct nonnegativity certificates from circuit-like structures rather than semidefinite moment matrices. Irene's pipeline computes that geometry first, then injects it into the constrained GP model.

8.7 Repository Anchors

1. `examples/SONCExample.py`: minimal SONC run path.
2. `examples/SONCExample33.py`: Section 3.3-style benchmark trace.
3. `tests/test_sonc_section3.py`: checks for barycentric weights, setup, and solve behavior.

CHAPTER

NINE

APPROXIMATING OPTIMUM VALUE

In various cases, separating functions and symbols are either very difficult or impossible. For example $x, \sin x$ and e^x are not algebraically independent, but their dependency can not be easily expressed in finitely many relations. One possible approach to these problems is replacing transcendental terms with a reasonably good approximation. This certainly will introduce more numerical error, but at least gives a reliable estimate for the optimum value.

9.1 Using `pyProximation.OrthSystem`

A simple and common method to approximate transcendental functions is using truncated Taylor expansions. In spite of its simplicity, there are various pitfalls which needs to be avoided. The most common is that the radius of convergence of the Taylor expansion may be smaller than the feasibility region of the optimization problem.

9.1.1 Example 1:

Find the minimum of $x + e^{x \sin x}$ *where* $-\pi \leq x \leq \pi$.

The objective function includes terms of x and transcendental functions. So, it is difficult to find a suitable algebraic representation to transform this optimization problem. Let us try to use Taylor expansion of $e^{x \sin x}$ to find an approximation for the optimum and compare the result with `scipy.optimize`, `pyOpt.ALPSO` and `pyOpt.NSGA2`:

```
from sympy import *
from Irene import *
# introduce symbols and functions
x = Symbol('x')
e = Function('e')(x)
# transcendental term of objective
f = exp(x * sin(x))
# Taylor expansion
f_app = f.series(x, 0, 12).removeO()
# initiate the Relaxation object
Rlx = SDPRelaxations([x])
# set the objective
Rlx.SetObjective(x + f_app)
# add support constraints
Rlx.AddConstraint(pi**2 - x**2 >= 0)
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
# using scipy
```

(continues on next page)

(continued from previous page)

```
from scipy.optimize import minimize
fun = lambda x: x[0] + exp(x[0] * sin(x[0]))
cons = (
    {'type': 'ineq', 'fun': lambda x: pi**2 - x[0]**2},
)
sol1 = minimize(fun, (0, 0), method='COBYLA', constraints=cons)
sol2 = minimize(fun, (0, 0), method='SLSQP', constraints=cons)
print "solution according to 'COBYLA':"
print sol1
print "solution according to 'SLSQP':"
print sol2

# pyOpt
from pyOpt import *


def objfunc(x):
    from numpy import sin, exp, pi
    f = x[0] + exp(x[0] * sin(x[0]))
    g = [
        x[0]**2 - pi**2
    ]
    fail = 0
    return f, g, fail

opt_prob = Optimization('A mixed function', objfunc)
opt_prob.addVar('x1', 'c', lower=-pi, upper=pi, value=0.0)
opt_prob.addObj('f')
opt_prob.addCon('g1', 'i')
# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

The output will look like:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 0.270121097565 seconds
              Run Time: 0.012974 seconds
Primal Objective Value: -416.628918881
  Dual Objective Value: -416.628917197
Feasible solution for moments of order 5

solution according to 'COBYLA':
     fun: 0.76611902154788758
   maxcv: 0.0
```

(continues on next page)

(continued from previous page)

```
 message: 'Optimization terminated successfully.'
    nfev: 34
  status: 1
 success: True
       x: array([ -4.42161128e-01,  -9.76206736e-05])
solution according to 'SLSQP':
     fun: 0.766119450232887
     jac: array([ 0.00154828,  0.        ,  0.        ])
 message: 'Optimization terminated successfully.'
    nfev: 17
     nit: 4
    njev: 4
  status: 0
 success: True
       x: array([-0.44164406,  0.        ])

ALPSO Solution to A mixed function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.0683
    Total Function Evaluations:      1240
    Lambda:     [ 0.]
    Seed: 1482112089.31088901

    Objectives:
        Name        Value        Optimum
             f        -2.14159             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------



NSGA-II Solution to A mixed function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.4231
    Total Function Evaluations:
```

(continues on next page)

(continued from previous page)

```
    Objectives:
        Name        Value        Optimum
             f        -2.14159             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                      Bounds
             g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------
```

Now instead of Taylor expansion, we use Legendre polynomials to estimate $e^{x \sin x}$. To find Legendre estimators, we use pyProximation which implements general Hilbert space methods (see Appendix-*pyProximation*):

```
from sympy import *
from Irene import *
from pyProximation import OrthSystem
# introduce symbols and functions
x = Symbol('x')
e = Function('e')(x)
# transcendental term of objective
f = exp(x * sin(x))
# Legendre polynomials via pyProximation
D = [(-pi, pi)]
S = OrthSystem([x], D)
# set B = {1, x, x^2, ..., x^12}
B = S.PolyBasis(12)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# extract the coefficients of approximation
Coeffs = S.Series(f)
# form the approximation
f_app = sum([S.OrthBase[i] * Coeffs[i] for i in range(len(S.OrthBase))])
# initiate the Relaxation object
Rlx = SDPRelaxations([x])
# set the objective
Rlx.SetObjective(x + f_app)
# add support constraints
Rlx.AddConstraint(pi**2 - x**2 >= 0)
# set the solver
Rlx.SetSDPSolver('dsdp')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
```

The output will be:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 0.722383022308 seconds
              Run Time: 0.077674 seconds
Primal Objective Value: -2.26145824829
  Dual Objective Value: -2.26145802066
Feasible solution for moments of order 6
```

By a small modification of the above code, we can employ Chebyshev polynomials for approximation:

```
from sympy import *
from Irene import *
from pyProximation import Measure, OrthSystem
# introduce symbols and functions
x = Symbol('x')
e = Function('e')(x)
# transcendental term of objective
f = exp(x * sin(x))
# Chebyshev polynomials via pyProximation
D = [(-pi, pi)]
# the Chebyshev weight
w = lambda x: 1. / sqrt(pi**2 - x**2)
M = Measure(D, w)
S = OrthSystem([x], D)
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^12}
B = S.PolyBasis(12)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# extract the coefficients of approximation
Coeffs = S.Series(f)
# form the approximation
f_app = sum([S.OrthBase[i] * Coeffs[i] for i in range(len(S.OrthBase))])
# initiate the Relaxation object
Rlx = SDPRelaxations([x])
# set the objective
Rlx.SetObjective(x + f_app)
# add support constraints
Rlx.AddConstraint(pi**2 - x**2 >= 0)
# set the solver
Rlx.SetSDPSolver('dsdp')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
```

which returns:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 0.805300951004 seconds
              Run Time: 0.066767 seconds
Primal Objective Value: -2.17420785198
  Dual Objective Value: -2.17420816422
Feasible solution for moments of order 6
```

This gives a better approximation for the optimum value. The optimum values found via Legendre and Chebyshev polynomials are certainly better than Taylor expansion and the results of `scipy.optimize`.

9.1.2 Example 2:

Find the minimum of $x \sinh y + e^{y \sin x}$ *where* $-\pi \leq x, y \leq \pi$.

Again, we use Legendre approximations for $\sinh y$ and $e^{y \sin x}$:

```
from sympy import *
from Irene import *
from pyProximation import OrthSystem
# introduce symbols and functions
x = Symbol('x')
y = Symbol('y')
sh = Function('sh')(y)
ch = Function('ch')(y)
# transcendental term of objective
f = exp(y * sin(x))
g = sinh(y)
# Legendre polynomials via pyProximation
D_f = [(-pi, pi), (-pi, pi)]
D_g = [(-pi, pi)]
Orth_f = OrthSystem([x, y], D_f)
Orth_g = OrthSystem([y], D_g)
# set bases
B_f = Orth_f.PolyBasis(10)
B_g = Orth_g.PolyBasis(10)
# link B_f to Orth_f and B_g to Orth_g
Orth_f.Basis(B_f)
Orth_g.Basis(B_g)
# generate the orthonormal bases
Orth_f.FormBasis()
Orth_g.FormBasis()
# extract the coefficients of approximations
Coeffs_f = Orth_f.Series(f)
Coeffs_g = Orth_g.Series(g)
# form the approximations
f_app = sum([Orth_f.OrthBase[i] * Coeffs_f[i]
             for i in range(len(Orth_f.OrthBase))])
g_app = sum([Orth_g.OrthBase[i] * Coeffs_g[i]
             for i in range(len(Orth_g.OrthBase))])
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y])
```

(continues on next page)

(continued from previous page)

```
# set the objective
Rlx.SetObjective(x * g_app + f_app)
# add support constraints
Rlx.AddConstraint(pi**2 - x**2 >= 0)
Rlx.AddConstraint(pi**2 - y**2 >= 0)
# set the sdp solver
Rlx.SetSDPSolver('cvxopt')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
# using scipy
from scipy.optimize import minimize
fun = lambda x: x[0] * sinh(x[1]) + exp(x[1] * sin(x[0]))
cons = (
    {'type': 'ineq', 'fun': lambda x: pi**2 - x[0]**2},
    {'type': 'ineq', 'fun': lambda x: pi**2 - x[1]**2}
)
sol1 = minimize(fun, (0, 0), method='COBYLA', constraints=cons)
sol2 = minimize(fun, (0, 0), method='SLSQP', constraints=cons)
print "solution according to 'COBYLA':"
print sol1
print "solution according to 'SLSQP':"
print sol2

# pyOpt
from pyOpt import *


def objfunc(x):
    from numpy import sin, sinh, exp, pi
    f = x[0] * sinh(x[1]) + exp(x[1] * sin(x[0]))
    g = [
        x[0]**2 - pi**2,
        x[1]**2 - pi**2
    ]
    fail = 0
    return f, g, fail

opt_prob = Optimization(
    'A trigonometric-hyperbolic-exponential function', objfunc)
opt_prob.addVar('x1', 'c', lower=-pi, upper=pi, value=0.0)
opt_prob.addVar('x2', 'c', lower=-pi, upper=pi, value=0.0)
opt_prob.addObj('f')
opt_prob.addCon('g1', 'i')
opt_prob.addCon('g2', 'i')
# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
```

(continues on next page)

(continued from previous page)

```
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

The result will be:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 4.09241986275 seconds
              Run Time: 0.123869 seconds
Primal Objective Value: -35.3574475835
  Dual Objective Value: -35.3574473266
Feasible solution for moments of order 5

solution according to 'COBYLA':
     fun: 1.0
   maxcv: 0.0
 message: 'Optimization terminated successfully.'
    nfev: 13
  status: 1
 success: True
       x: array([ 0.,  0.])
solution according to 'SLSQP':
     fun: 1
     jac: array([ 0.,  0.,  0.])
 message: 'Optimization terminated successfully.'
    nfev: 4
     nit: 1
    njev: 1
  status: 0
 success: True
       x: array([ 0.,  0.])

ALPSO Solution to A trigonometric-hyperbolic-exponential function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.0946
    Total Function Evaluations:      1240
    Lambda: [ 0.  0.]
    Seed: 1482112613.82665610

    Objectives:
        Name        Value        Optimum
             f        -35.2814             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
```

(continues on next page)

(continued from previous page)

```
            x1       c      -3.141593      -3.14e+00     3.14e+00
            x2       c       3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
            g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00
            g2           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to A trigonometric-hyperbolic-exponential function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.5331
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
            f        -35.2814             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
            x1       c       3.141593      -3.14e+00     3.14e+00
            x2       c      -3.141593      -3.14e+00     3.14e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
            g1           i       -1.00e+21 <= 0.000000 <= 0.00e+00
            g2           i       -1.00e+21 <= 0.000000 <= 0.00e+00

--------------------------------------------------------------------------------
```

which shows a significant improvement compare to results of `scipi.minimize`.

CHAPTER

TEN

BENCHMARK OPTIMIZATION PROBLEMS

There are benchmark problems to evaluated how good an optimization method works. We apply the generalized relaxation method to some of these benchmarks that are mainly taken from [MJXY].

10.1 Rosenbrock Function

The original Rosenbrock function is $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$ which is a sums of squares and attains its minimum at $(1, 1)$. The global minimum is inside a long, narrow, parabolic shaped flat valley. To find the valley is trivial. To converge to the global minimum, however, is difficult. The same holds for a generalized form of Rosenbrock function which is defined as:

$$f(x_1, \ldots, x_n) = \sum_{i=1}^{n-1} 100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2.$$

Since f is a sum of squares, and $f(1, \ldots, 1) = 0$, the global minimum is equal to 0. The following code compares various optimization methods including the relaxation method, to find a minimum for f where $9 - x_i^2 \geq 0$ for $i = 1, \ldots, 9$:

```
from sympy import *
from Irene import SDPRelaxations
NumVars = 9
# define the symbolic variables and functions
x = [Symbol('x%d' % i) for i in range(NumVars)]

print "Relaxation method:"
# initiate the SDPRelaxations object
Rlx = SDPRelaxations(x)
# Rosenbrock function
rosen = sum([100 * (x[i + 1] - x[i]**2)**2 + (1 - x[i])
             ** 2 for i in range(NumVars - 1)])
# set the objective
Rlx.SetObjective(rosen)
# add constraints
for i in range(NumVars):
    Rlx.AddConstraint(9 - x[i]**2 >= 0)
# set the sdp solver
Rlx.SetSDPSolver('cvxopt')
# initiate the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
```

(continues on next page)

(continued from previous page)

```
print Rlx.Solution
# solve with scipy
from scipy.optimize import minimize
fun = lambda x: sum([100 * (x[i + 1] - x[i]**2)**2 +
                     (1 - x[i])**2 for i in range(NumVars - 1)])
cons = [
    {'type': 'ineq', 'fun': lambda x: 9 - x[i]**2} for i in range(NumVars)]
x0 = tuple([0 for _ in range(NumVars)])
sol1 = minimize(fun, x0, method='COBYLA', constraints=cons)
sol2 = minimize(fun, x0, method='SLSQP', constraints=cons)

print "solution according to 'COBYLA':"
print sol1
print "solution according to 'SLSQP':"
print sol2

# pyOpt
from pyOpt import *


def objfunc(x):
    f = sum([100 * (x[i + 1] - x[i]**2)**2 + (1 - x[i])
             ** 2 for i in range(NumVars - 1)])
    g = [x[i]**2 - 9 for i in range(NumVars)]
    fail = 0
    return f, g, fail

opt_prob = Optimization(
    'The Rosenbrock function', objfunc)
opt_prob.addObj('f')
for i in range(NumVars):
    opt_prob.addVar('x%d' % (i + 1), 'c', lower=-3, upper=3, value=0.0)
    opt_prob.addCon('g%d' % (i + 1), 'i')

# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

The result is:

```
Relaxation method:
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 750.234924078 seconds
              Run Time: 8.43369 seconds
Primal Objective Value: 1.67774267808e-08
```

(continues on next page)

(continued from previous page)

```
  Dual Objective Value: 1.10015692778e-08
Feasible solution for moments of order 2

solution according to 'COBYLA':
     fun: 4.4963584556077389
   maxcv: 0.0
 message: 'Maximum number of function evaluations has been exceeded.'
    nfev: 1000
  status: 2
 success: False
       x: array([  8.64355944e-01,   7.47420978e-01,   5.59389194e-01,
         3.16212252e-01,   1.05034350e-01,   2.05923923e-02,
         9.44389237e-03,   1.12341021e-02,  -7.74530516e-05])
     fun: 1.3578865444308464e-07
     jac: array([ 0.00188377,  0.00581741, -0.00182463,  0.00776938, -0.00343305,
       -0.00186283,  0.0020364 ,  0.00881489, -0.0047164 ,  0.        ])
solution according to 'SLSQP':
 message: 'Optimization terminated successfully.'
    nfev: 625
     nit: 54
    njev: 54
  status: 0
 success: True
       x: array([ 1.00000841,  1.00001216,  1.00000753,  1.00001129,  1.00000134,
        1.00000067,  1.00000502,  1.00000682,  0.99999006])

ALPSO Solution to The Rosenbrock function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                   10.6371
    Total Function Evaluations:     48040
    Lambda: [ 0.  0.  0.  0.  0.  0.  0.  0.  0.]
    Seed: 1482114864.60097694

    Objectives:
        Name        Value        Optimum
             f        0.590722             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       0.992774      -3.00e+00     3.00e+00
             x2       c       0.986019      -3.00e+00     3.00e+00
             x3       c       0.970756      -3.00e+00     3.00e+00
             x4       c       0.942489      -3.00e+00     3.00e+00
             x5       c       0.886910      -3.00e+00     3.00e+00
             x6       c       0.787367      -3.00e+00     3.00e+00
             x7       c       0.618875      -3.00e+00     3.00e+00
             x8       c       0.382054      -3.00e+00     3.00e+00
```

(continues on next page)

(continued from previous page)

```
            x9       c       0.143717      -3.00e+00     3.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
            g1           i       -1.00e+21 <= -8.014399 <= 0.00e+00
            g2           i       -1.00e+21 <= -8.027767 <= 0.00e+00
            g3           i       -1.00e+21 <= -8.057633 <= 0.00e+00
            g4           i       -1.00e+21 <= -8.111714 <= 0.00e+00
            g5           i       -1.00e+21 <= -8.213391 <= 0.00e+00
            g6           i       -1.00e+21 <= -8.380053 <= 0.00e+00
            g7           i       -1.00e+21 <= -8.616994 <= 0.00e+00
            g8           i       -1.00e+21 <= -8.854035 <= 0.00e+00
            g9           i       -1.00e+21 <= -8.979345 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to The Rosenbrock function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    0.6244
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
            f          5.5654             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
            x1       c       0.727524      -3.00e+00     3.00e+00
            x2       c       0.537067      -3.00e+00     3.00e+00
            x3       c       0.296186      -3.00e+00     3.00e+00
            x4       c       0.094420      -3.00e+00     3.00e+00
            x5       c       0.017348      -3.00e+00     3.00e+00
            x6       c       0.009658      -3.00e+00     3.00e+00
            x7       c       0.015372      -3.00e+00     3.00e+00
            x8       c       0.009712      -3.00e+00     3.00e+00
            x9       c       0.001387      -3.00e+00     3.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
            g1           i       -1.00e+21 <= -8.470708 <= 0.00e+00
            g2           i       -1.00e+21 <= -8.711559 <= 0.00e+00
            g3           i       -1.00e+21 <= -8.912274 <= 0.00e+00
            g4           i       -1.00e+21 <= -8.991085 <= 0.00e+00
            g5           i       -1.00e+21 <= -8.999699 <= 0.00e+00
            g6           i       -1.00e+21 <= -8.999907 <= 0.00e+00
            g7           i       -1.00e+21 <= -8.999764 <= 0.00e+00
```

(continues on next page)

(continued from previous page)

```
            g8          i       -1.00e+21 <= -8.999906 <= 0.00e+00
            g9          i       -1.00e+21 <= -8.999998 <= 0.00e+00

--------------------------------------------------------------------------
```

The relaxation method returns values very close to the actual minimum but two out of other three methods fail to estimate the minimum correctly.

10.2 Giunta Function

Giunta is an example of continuous, differentiable, separable, scalable, multimodal function defined by:

$$\begin{aligned} f(x_1, x_2) &= \tfrac{3}{5} + \textstyle\sum_{i=1}^{2}[\sin(\tfrac{16}{15}x_i - 1) \\ &+ \sin^2(\tfrac{16}{15}x_i - 1) \\ &+ \tfrac{1}{50}\sin(4(\tfrac{16}{15}x_i - 1))]. \end{aligned}$$

The following code optimizes f when $1 - x_i^2 \geq 0$:

```
from sympy import *
from Irene import *
x = Symbol('x')
y = Symbol('y')
s1 = Symbol('s1')
c1 = Symbol('c1')
s2 = Symbol('s2')
c2 = Symbol('c2')
f = .6 + (sin(x - 1) + (sin(x - 1))**2 + .02 * sin(4 * (x - 1))) + \
    (sin(y - 1) + (sin(y - 1))**2 + .02 * sin(4 * (y - 1)))
f = expand(f, trig=True)
f = N(f.subs({sin(x): s1, cos(x): c1, sin(y): s2, cos(y): c2}))
rels = [s1**2 + c1**2 - 1, s2**2 + c2**2 - 1]
Rlx = SDPRelaxations([s1, c1, s2, c2], rels)
Rlx.SetObjective(f)
Rlx.AddConstraint(1 - s1**2 >= 0)
Rlx.AddConstraint(1 - s2**2 >= 0)
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
# solve with scipy
from scipy.optimize import minimize
fun = lambda x: .6 + (sin((16. / 15.) * x[0] - 1) + (sin((16. / 15.) * x[0] - 1))**2 + .
↪02 * sin(4 * ((16. / 15.) * x[0] - 1))) + (
    sin((16. / 15.) * x[1] - 1) + (sin((16. / 15.) * x[1] - 1))**2 + .02 * sin(4 * ((16.␣
↪/ 15.) * x[1] - 1)))
cons = [
    {'type': 'ineq', 'fun': lambda x: 1 - x[i]**2} for i in range(2)]
x0 = tuple([0 for _ in range(2)])
sol1 = minimize(fun, x0, method='COBYLA', constraints=cons)
sol2 = minimize(fun, x0, method='SLSQP', constraints=cons)
print "solution according to 'COBYLA':"
print sol1
```

(continues on next page)

(continued from previous page)

```
print "solution according to 'SLSQP':"
print sol2

# pyOpt
from pyOpt import *


def objfunc(x):
    f = .6 + (sin((16. / 15.) * x[0] - 1) + (sin((16. / 15.) * x[0] - 1))**2 + .02 *␣
↪sin(4 * ((16. / 15.) * x[0] - 1))) + (
        sin((16. / 15.) * x[1] - 1) + (sin((16. / 15.) * x[1] - 1))**2 + .02 * sin(4 *␣
↪((16. / 15.) * x[1] - 1)))
    g = [x[i]**2 - 1 for i in range(2)]
    fail = 0
    return f, g, fail

opt_prob = Optimization(
    'The Giunta function', objfunc)
opt_prob.addObj('f')
for i in range(2):
    opt_prob.addVar('x%d' % (i + 1), 'c', lower=-1, upper=1, value=0.0)
    opt_prob.addCon('g%d' % (i + 1), 'i')

# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

and the result is:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 2.53814482689 seconds
              Run Time: 0.041321 seconds
Primal Objective Value: 0.0644704534329
  Dual Objective Value: 0.0644704595475
Feasible solution for moments of order 2

solution according to 'COBYLA':
     fun: 0.064470430891900576
   maxcv: 0.0
 message: 'Optimization terminated successfully.'
    nfev: 40
  status: 1
 success: True
       x: array([ 0.46730658,  0.4674184 ])
solution according to 'SLSQP':
```

(continues on next page)

(continued from previous page)

```
     fun: 0.0644704633430450
     jac: array([-0.00029983, -0.00029983,  0.        ])
 message: 'Optimization terminated successfully.'
    nfev: 13
     nit: 3
    njev: 3
  status: 0
 success: True
       x: array([ 0.46717727,  0.46717727])

ALPSO Solution to The Giunta function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                   10.6180
    Total Function Evaluations:      1240
    Lambda: [ 0.  0.]
    Seed: 1482115204.08583212

    Objectives:
        Name        Value        Optimum
             f       0.0644704             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c       0.467346      -1.00e+00     1.00e+00
             x2       c       0.467369      -1.00e+00     1.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= -0.781588 <= 0.00e+00
             g2           i       -1.00e+21 <= -0.781566 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to The Giunta function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                   50.9196
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
             f       0.0644704             0
```

(continues on next page)

(continued from previous page)

```
        Variables (c - continuous, i - integer, d - discrete):
        Name    Type        Value       Lower Bound  Upper Bound
             x1       c       0.467403      -1.00e+00     1.00e+00
             x2       c       0.467324      -1.00e+00     1.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= -0.781535 <= 0.00e+00
             g2           i       -1.00e+21 <= -0.781608 <= 0.00e+00

--------------------------------------------------------------------------------
```

10.3 Parsopoulos Function

Parsopoulos is defined as $f(x, y) = \cos^2(x) + \sin^2(y)$. The following code computes its minimum where $-5 \leq x, y \leq 5$:

```
from sympy import *
from Irene import *
x = Symbol('x')
y = Symbol('y')
s1 = Symbol('s1')
c1 = Symbol('c1')
s2 = Symbol('s2')
c2 = Symbol('c2')
f = c1**2 + s2**2
rels = [s1**2 + c1**2 - 1, s2**2 + c2**2 - 1]
Rlx = SDPRelaxations([s1, c1, s2, c2], rels)
Rlx.SetObjective(f)
Rlx.AddConstraint(1 - s1**2 >= 0)
Rlx.AddConstraint(1 - s2**2 >= 0)
Rlx.MomentsOrd(2)
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
# solve with scipy
from scipy.optimize import minimize
fun = lambda x: cos(x[0])**2 + sin(x[1])**2
cons = [
    {'type': 'ineq', 'fun': lambda x: 25 - x[i]**2} for i in range(2)]
x0 = tuple([0 for _ in range(2)])
sol1 = minimize(fun, x0, method='COBYLA', constraints=cons)
sol2 = minimize(fun, x0, method='SLSQP', constraints=cons)
print "solution according to 'COBYLA':"
print sol1
print "solution according to 'SLSQP':"
print sol2

# pyOpt
```

(continues on next page)

(continued from previous page)

```
from pyOpt import *


def objfunc(x):
    f = cos(x[0])**2 + sin(x[1])**2
    g = [x[i]**2 - 25 for i in range(2)]
    fail = 0
    return f, g, fail

opt_prob = Optimization(
    'The Parsopoulos function', objfunc)
opt_prob.addObj('f')
for i in range(2):
    opt_prob.addVar('x%d' % (i + 1), 'c', lower=-5, upper=5, value=0.0)
    opt_prob.addCon('g%d' % (i + 1), 'i')

# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

which returns:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 2.48692297935 seconds
              Run Time: 0.035358 seconds
Primal Objective Value: -3.74719295193e-10
  Dual Objective Value: 5.43053240402e-12
Feasible solution for moments of order 2

solution according to 'COBYLA':
     fun: 1.83716742579312e-08
   maxcv: 0.0
 message: 'Optimization terminated successfully.'
    nfev: 35
  status: 1
 success: True
       x: array([  1.57072551e+00,   1.15569800e-04])
solution according to 'SLSQP':
     fun: 1
     jac: array([ -1.49011612e-08,   1.49011612e-08,   0.00000000e+00])
 message: 'Optimization terminated successfully.'
    nfev: 4
     nit: 1
    njev: 1
  status: 0
```

(continues on next page)

(continued from previous page)

```
 success: True
       x: array([ 0.,  0.])

ALPSO Solution to The Parsopoulos function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                    4.4576
    Total Function Evaluations:      1240
    Lambda: [ 0.  0.]
    Seed: 1482115438.17070389

    Objectives:
        Name        Value        Optimum
             f     5.68622e-09             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -4.712408      -5.00e+00     5.00e+00
             x2       c      -0.000073      -5.00e+00     5.00e+00

        Constraints (i - inequality, e - equality):
        Name    Type                    Bounds
             g1           i       -1.00e+21 <= -2.793212 <= 0.00e+00
             g2           i       -1.00e+21 <= -25.000000 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to The Parsopoulos function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                   17.7197
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
             f     2.37167e-08             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -1.570676      -5.00e+00     5.00e+00
             x2       c       3.141496      -5.00e+00     5.00e+00

        Constraints (i - inequality, e - equality):
```

(continues on next page)

(continued from previous page)

```
        Name     Type                    Bounds
             g1            i       -1.00e+21 <= -22.532977 <= 0.00e+00
             g2            i       -1.00e+21 <= -15.131000 <= 0.00e+00

--------------------------------------------------------------------------------
```

10.4 Shubert Function

Shubert function is defined by:

$$f(x_1, \ldots, x_n) = \prod_{i=1}^{n} \left(\sum_{j=1}^{5} \cos((j+1)x_i + i) \right).$$

It is a continuous, differentiable, separable, non-scalable, multimodal function. The following code compares the result of five optimizers when $-10 \leq x_i \leq 10$ and $n = 2$:

```
from sympy import *
from Irene import *
x = Symbol('x')
y = Symbol('y')
s1 = Symbol('s1')
c1 = Symbol('c1')
s2 = Symbol('s2')
c2 = Symbol('c2')
f = sum([cos((j + 1) * x + j) for j in range(1, 6)]) * \
    sum([cos((j + 1) * y + j) for j in range(1, 6)])
obj = N(expand(f, trig=True).subs(
    {sin(x): s1, cos(x): c1, sin(y): s2, cos(y): c2}))
rels = [s1**2 + c1**2 - 1, s2**2 + c2**2 - 1]
Rlx = SDPRelaxations([s1, c1, s2, c2], rels)
Rlx.SetObjective(obj)
Rlx.AddConstraint(1 - s1**2 >= 0)
Rlx.AddConstraint(1 - s2**2 >= 0)
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
g = lambda x: sum([cos((j + 1) * x[0] + j) for j in range(1, 6)]) * \
    sum([cos((j + 1) * x[1] + j) for j in range(1, 6)])
x0 = (-5, 5)
from scipy.optimize import minimize
cons = (
    {'type': 'ineq', 'fun': lambda x: 100 - x[0]**2},
    {'type': 'ineq', 'fun': lambda x: 100 - x[1]**2})
sol1 = minimize(g, x0, method='COBYLA', constraints=cons)
sol2 = minimize(g, x0, method='SLSQP', constraints=cons)
print "solution according to 'COBYLA':"
print sol1
print "solution according to 'SLSQP':"
print sol2
```

(continues on next page)

(continued from previous page)

```
from sage.all import *
m1 = minimize_constrained(g, cons=[cn['fun'] for cn in cons], x0=x0)
m2 = minimize_constrained(g, cons=[cn['fun']
                                   for cn in cons], x0=x0, algorithm='l-bfgs-b')
print "Sage:"
print "minimize_constrained (default):", m1, g(m1)
print "minimize_constrained (l-bfgs-b):", m2, g(m2)

# pyOpt
from pyOpt import *


def objfunc(x):
    f = sum([cos((j + 1) * x[0] + j) for j in range(1, 6)]) * \
        sum([cos((j + 1) * x[1] + j) for j in range(1, 6)])
    g = [x[i]**2 - 100 for i in range(2)]
    fail = 0
    return f, g, fail

opt_prob = Optimization(
    'The Shubert function', objfunc)
opt_prob.addObj('f')
for i in range(2):
    opt_prob.addVar('x%d' % (i + 1), 'c', lower=-10, upper=10, value=0.0)
    opt_prob.addCon('g%d' % (i + 1), 'i')

# Augmented Lagrangian Particle Swarm Optimizer
alpso = ALPSO()
alpso(opt_prob)
print opt_prob.solution(0)
# Non Sorting Genetic Algorithm II
nsg2 = NSGA2()
nsg2(opt_prob)
print opt_prob.solution(1)
```

The result is:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 730.02412415 seconds
              Run Time: 5.258507 seconds
Primal Objective Value: -18.0955649723
  Dual Objective Value: -18.0955648855
Feasible solution for moments of order 6
Scipy 'COBYLA':
     fun: -3.3261182321238367
   maxcv: 0.0
 message: 'Optimization terminated successfully.'
    nfev: 39
  status: 1
 success: True
```

(continues on next page)

(continued from previous page)

```
       x: array([-3.96201407,  4.81176624])
Scipy 'SLSQP':
     fun: -0.856702387212005
     jac: array([-0.00159422,  0.00080796,  0.        ])
 message: 'Optimization terminated successfully.'
    nfev: 35
     nit: 7
    njev: 7
  status: 0
 success: True
       x: array([-4.92714381,  4.81186391])
Sage:
minimize_constrained (default): (-3.962032420336303, 4.811734682897321) -3.32611819422
minimize_constrained (l-bfgs-b): (-3.962032420336303, 4.811734682897321) -3.32611819422

ALPSO Solution to The Shubert function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                   37.7526
    Total Function Evaluations:      2200
    Lambda: [ 0.  0.]
    Seed: 1482115770.57303905

    Objectives:
        Name        Value        Optimum
             f        -18.0956             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -7.061398      -1.00e+01     1.00e+01
             x2       c      -1.471424      -1.00e+01     1.00e+01

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= -50.136654 <= 0.00e+00
             g2           i       -1.00e+21 <= -97.834910 <= 0.00e+00

--------------------------------------------------------------------------------


NSGA-II Solution to The Shubert function
================================================================================

        Objective Function: objfunc

    Solution:
--------------------------------------------------------------------------------
    Total Time:                   97.6291
```

(continues on next page)

(continued from previous page)

```
    Total Function Evaluations:

    Objectives:
        Name        Value        Optimum
             f        -18.0955             0

        Variables (c - continuous, i - integer, d - discrete):
        Name    Type       Value       Lower Bound  Upper Bound
             x1       c      -0.778010      -1.00e+01     1.00e+01
             x2       c      -7.754277      -1.00e+01     1.00e+01

        Constraints (i - inequality, e - equality):
        Name    Type                     Bounds
             g1           i       -1.00e+21 <= -99.394700 <= 0.00e+00
             g2           i       -1.00e+21 <= -39.871193 <= 0.00e+00

--------------------------------------------------------------------------------
```

We note that four out of six other optimizers stuck at a local minimum and return incorrect values.

Moreover, we employed 20 different optimizers included in pyOpt and only 4 of them returned the correct optimum value.

10.5 McCormick Function

McCormick function is defined by

$$f(x, y) = \sin(x + y) + (x - y)^2 - 1.5x + 2.5y + 1.$$

Attains its minimum at $f(-.54719, -1.54719) \approx -1.9133$:

```
from sympy import *
from Irene import *
from pyProximation import OrthSystem
# introduce symbols
x = Symbol('x')
y = Symbol('y')
z = Symbol('z')
# transcendental term of objective
f = sin(z)
# Legendre polynomials via pyProximation
D_f = [(-2, 2)]
Orth_f = OrthSystem([z], D_f)
# set bases
B_f = Orth_f.PolyBasis(10)
# link B_f to Orth_f
Orth_f.Basis(B_f)
# generate the orthonormal bases
Orth_f.FormBasis()
# extract the coefficients of approximations
Coeffs_f = Orth_f.Series(f)
# form the approximations
```

(continues on next page)

(continued from previous page)

```
f_app = sum([Orth_f.OrthBase[i] * Coeffs_f[i]
             for i in range(len(Orth_f.OrthBase))])
# objective function
obj = f_app.subs({z: x + y}) + (x - y)**2 - 1.5 * x + 2.5 * y + 1
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y])
# set the objective
Rlx.SetObjective(obj)
# add support constraints
Rlx.AddConstraint(4 - (x**2 + y**2) >= 0)
# set the sdp solver
Rlx.SetSDPSolver('cvxopt')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
Rlx.Solution.ExtractSolution('lh',1)
print Rlx.Solution
```

Results in:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 10.6071600914 seconds
              Run Time: 0.070002 seconds
Primal Objective Value: -1.91322353633
  Dual Objective Value: -1.91322352558
               Support:
                (-0.54724056855672309, -1.5473099043318805)
        Support solver: Lasserre--Henrion
Feasible solution for moments of order 5
```

10.6 Schaffer Function N.2

Schaffer function N.2 is

$$f(x,y) = \frac{\sin^2(x^2-y^2)-.5}{(1+.001(x^2+y^2))^2}.$$

Attains its minimum at $f(0,0) = .5$:

```
from sympy import *
from Irene import *
from pyProximation import OrthSystem, Measure
# introduce symbols and functions
x = Symbol('x')
y = Symbol('y')
z = Symbol('z')
# transcendental term of objective
f = (sin(z))**2
# Chebyshev polynomials via pyProximation
```

(continues on next page)

(continued from previous page)

```
D_f = [(-2, 2)]
w = lambda x: 1. / sqrt(4 - x**2)
M = Measure(D_f, w)
# link the measure to S
Orth_f = OrthSystem([z], D_f)
Orth_f.SetMeasure(M)
# set bases
B_f = Orth_f.PolyBasis(8)
# link B to S
Orth_f.Basis(B_f)
# generate the orthonormal bases
Orth_f.FormBasis()
# extract the coefficients of approximations
Coeffs_f = Orth_f.Series(f)
# form the approximations
f_app = sum([Orth_f.OrthBase[i] * Coeffs_f[i]
             for i in range(len(Orth_f.OrthBase))])
# objective function
obj = f_app.subs({z: x**2 - y**2}) - .5
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y])
# settings
Rlx.Probability = False
# set the objective
Rlx.SetObjective(obj)
# add support constraints
Rlx.AddConstraint(4 - (x**2 + y**2) >= 0)
# moment constraint
Rlx.MomentConstraint(Mom((1 + .001 * (x**2 + y**2)**2)) == 1)
# set the sdp solver
Rlx.SetSDPSolver('cvxopt')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
Rlx.Solution.ExtractSolution('lh', 1)
print Rlx.Solution
```

The result:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
                Status: Optimal
   Initialization Time: 26.6285181046 seconds
              Run Time: 0.110288 seconds
Primal Objective Value: -0.495770329702
  Dual Objective Value: -0.495770335895
               Support:
                (1.3348173524856991e-15, 8.3700760032311997e-17)
        Support solver: Lasserre--Henrion
Feasible solution for moments of order 6
```

10.7 Schaffer Function N.4

Schaffer function N.4 is

$$f(x,y) = \frac{\cos^2(\sin(|(x^2-y^2)|)) - .5}{(1+.001(x^2+y^2))^2}.$$

The minimum value is -0.207421:

```
from sympy import *
from Irene import *
from pyProximation import OrthSystem, Measure
# introduce symbols and functions
x = Symbol('x')
y = Symbol('y')
z = Symbol('z')
# transcendental term of objective
f = (cos(sin(abs(z))))**2
# Chebyshev polynomials via pyProximation
D_f = [(-2, 2)]
w = lambda x: 1. / sqrt(4 - x**2)
M = Measure(D_f, w)
# link the measure to S
Orth_f = OrthSystem([z], D_f)
Orth_f.SetMeasure(M)
# set bases
B_f = Orth_f.PolyBasis(12)
# link B_f to Orth_f
Orth_f.Basis(B_f)
# generate the orthonormal bases
Orth_f.FormBasis()
# extract the coefficients of approximations
Coeffs_f = Orth_f.Series(f)
# form the approximations
f_app = sum([Orth_f.OrthBase[i] * Coeffs_f[i]
             for i in range(len(Orth_f.OrthBase))])
# objective function
obj = f_app.subs({z: x**2 - y**2}) - .5
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y])
# settings
Rlx.Probability = False
# set the objective
Rlx.SetObjective(obj)
# add support constraints
Rlx.AddConstraint(4 - (x**2 + y**2) >= 0)
# moment constraint
Rlx.MomentConstraint(Mom((1 + .001 * (x**2 + y**2)**2)) == 1)
# set the sdp solver
Rlx.SetSDPSolver('csdp')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
print Rlx.Solution
```

Result is:

```
Solution of a Semidefinite Program:
                Solver: DSDP
                Status: Optimal
   Initialization Time: 497.670987129 seconds
              Run Time: 75.423031 seconds
Primal Objective Value: -0.203973186683
  Dual Objective Value: -0.208094722977
Feasible solution for moments of order 12
```

10.8 Drop-Wave Function

The Drop-Wave function is multimodal and highly complex:

$$f(x,y) = -\frac{1+\cos(12\sqrt{x^2+y^2})}{.5(x^2+y^2)+2}.$$

It has a global minimum at $f(0,0) = -1$. We use Bhaskara's approximation $\cos(x) \approx \frac{\pi^2-4x^2}{\pi^2+x^2}$ to solve this problem:

```
from sympy import *
from Irene import *
# introduce symbols and functions
x = Symbol('x')
y = Symbol('y')
# objective function
obj = -((pi**2 + 12**2 * (x**2 + y**2)) + (pi**2 - 4 * 12**2 * (x**2 + y**2))
        ) / (((pi**2 + 12**2 * (x**2 + y**2))) * (2 + .5 * (x**2 + y**2)))
# numerator
top = numer(obj)
# denominator
bot = expand(denom(obj))
# initiate the Relaxation object
Rlx = SDPRelaxations([x, y])
# settings
Rlx.Probability = False
# set the objective
Rlx.SetObjective(top)
# moment constraint
Rlx.MomentConstraint(Mom(bot) == 1)
# set the sdp solver
Rlx.SetSDPSolver('cvxopt')
# initialize the SDP
Rlx.InitSDP()
# solve the SDP
Rlx.Minimize()
Rlx.Solution.ExtractSolution('lh', 1)
print Rlx.Solution
```

The output is:

```
Solution of a Semidefinite Program:
                Solver: CVXOPT
```

(continues on next page)

(continued from previous page)

```
                Status: Optimal
   Initialization Time: 0.0663878917694 seconds
              Run Time: 0.005548 seconds
Primal Objective Value: -1.00000132341
  Dual Objective Value: -1.00000123991
               Support:
                (-0.0, 0.0)
        Support solver: Lasserre--Henrion
Feasible solution for moments of order 1
```

CHAPTER

ELEVEN

EXAMPLES AND VALIDATION

This chapter lists runnable entry points that exercise the three method families.

11.1 Recommended Example Sequence

1. `examples/Example01.py` for SDP hierarchy flow.
2. `examples/GPExample.py` for geometric relaxation flow.
3. `examples/SONCExample.py` and `examples/SONCExample33.py` for SONC flow.

11.2 API Quick Reference

Script	Primary classes	Solver dependency
`examples/Example01.py`	`SDPRelaxations`	SDP solver (for example `csdp`, `sdpa`, `dsdp`, or `cvxopt`)
`examples/GPExample.py`	`OptimizationProblem`, `GPRelaxations`	`gpkit` backend
`examples/SONCExample.py`	`OptimizationProblem`, `SONCRelaxations`	`gpkit` backend
`examples/SONCExample33.py`	`OptimizationProblem`, `SONCRelaxations`	`gpkit` backend

11.3 SDP Example

Run:

```
python examples/Example01.py
```

Expected behavior:

1. Initializes an `SDPRelaxations` object with symbolic relations.
2. Solves an SDP lower-bound problem via selected solver.
3. Prints solver summary and objective values.

11.4 Geometric Programming Example

Run:

```
python examples/GPExample.py
```

Expected behavior:

1. Builds an `OptimizationProblem` from semigroup-algebra expressions.
2. Constructs a `GPRelaxations` model.
3. Prints transformation matrix information and GP solution details.

11.5 SONC Examples

Run:

```
python examples/SONCExample.py
python examples/SONCExample33.py
```

Expected behavior:

1. Builds constrained SONC models from semigroup-algebra expressions.
2. Prints a lower bound when solver/model setup succeeds.
3. Reports runtime solver status if GP solving is not available in the current environment.

11.6 Example 3.3 Traceability

The script `examples/SONCExample33.py` is aligned with the Section 3.3 benchmark used in the repository and is paired with checks in `tests/test_sonc_section3.py`.

11.7 Regression Validation

Run the test suite from the repository root:

```
python -m unittest discover tests/
```

This is the recommended consistency check after modifying optimization modules or documentation examples.

CHAPTER

TWELVE

CODE DOCUMENTATION

This is the base module for all other objects of the package.

- *LaTeX* returns a LaTeX string out of an *Irene* object.
- *base* is the parent of all *Irene* objects.

`Irene.base.LaTeX(obj)`

Returns LaTeX representation of Irene's objects.

class `Irene.base.base`

All the modules in *Irene* extend this class which perform some common tasks such as checking existence of certain software.

AvailableSDPSolvers()

find the existing sdp solvers.

static which(*program*)

Check the availability of the *program* system-wide. Returns the path of the program if exists and returns 'None' otherwise.

This module is responsible for conversion of a given symbolic optimization problem into semidefinite optimization problems. The main classes included in this module are:

- *SDPRelaxations*
- *SDRelaxSol*
- *Mom*

`Irene.relaxations.Calpha_(expn, Mmnt)`

Given an exponent *expn*, this function finds the corresponding C_{expn} matrix which can be used for parallel processing.

`Irene.relaxations.Calpha__(expn, Mmnt, ii, q)`

Given an exponent *expn*, this function finds the corresponding C_{expn} matrix which can be used for parallel processing.

class `Irene.relaxations.Mom(expr)`

This is a simple interface to define moment constraints to be used via *SDPRelaxations.MomentConstraint*. It takes a sympy expression as input and initiates an object which can be used to force particular constraints on the moment sequence.

Example: Force the moment of $x^2 f(x) + f(x)^2$ to be at least *.5*:

```
Mom(x**2 * f + f**2) >= .5
# OR
Mom(x**2 * f) + Mom(f**2) >= .5
```

class `Irene.relaxations.SDPRelaxations`(*gens*, *relations=()*, *name='SDPRlx'*)

This class defines a function space by taking a family of sympy symbolic functions and relations among them. Simply, it initiates a commutative free real algebra on the symbolic functions and defines the function space as the quotient of the free algebra by the ideal generated by the given relations. It takes three arguments:

- *gens* which is a list of `sympy` symbols and function symbols,
- *relations* which is a set of `sympy` expressions in terms of *gens* that defines an ideal.
- *name* is a given name which is used to save the state of the instant at break.

AddConstraint(*cnst*)

Takes an (in)equality as an algebraic combination of the generating functions that defines the feasibility region. It reduces the defining (in)equalities according to the given relations.

Calpha(*expn*, *Mmnt*)

Given an exponent *expn*, this method finds the corresponding C_{expn} matrix.

Commit(*blk*, *c*, *idx*)

Sets the latest computed values for the final SDP and saves the current state.

Decompose()

Returns a dictionary that associates a list to every constraint, $g_i \geq 0$ for $i = 0, \ldots, m$, where $g_0 = 1$. Each list consists of elements of algebra whose sums of squares is equal to σ_i and $f - f_* = \sum_{i=0}^{m} \sigma_i g_i$. Here, f_* is the output of the `SDPRelaxation.Minimize()`.

ExponentsVec(*deg*)

Returns all the exponents that appear in the reduced basis of all monomials of the auxiliary symbols of degree at most *deg*.

InitSDP()

Initializes the SDP based on the value of `self.Parallel`. If it is `True`, it runs in parallel mode, otherwise in serial.

LocalizedMoment(*p*)

Computes the reduced symbolic moment generating matrix localized at *p*.

LocalizedMoment_(*p*)

Computes the reduced symbolic moment generating matrix localized at *p*.

Minimize()

Finds the minimum of the truncated moment problem which provides a lower bound for the actual minimum.

MomentConstraint(*cnst*)

Takes constraints on the moments. The input must be an instance of *Mom* class.

MomentMat()

Returns the numerical moment matrix resulted from solving the SDP.

MomentsOrd(*ordr*)

Sets the order of moments to be considered.

PolyCoefFullVec()

return the vector of coefficient of the reduced objective function as an element of the vector space of elements of degree up to the order of moments.

ReduceExp(*expr*)

Takes an expression *expr*, either in terms of internal free symbolic variables or generating functions and returns the reduced expression in terms of internal symbolic variables, if a relation among generators is present, otherwise it just substitutes generating functions with their corresponding internal symbols.

ReducedMonomialBase(*deg*)

Returns a reduce monomial basis up to degree *d*.

RelaxationDeg()

Finds the minimum required order of moments according to user's request, objective function and constraints.

Resume()

Resumes the process of a previously saved program.

SaveState()

Saves the current state of the relaxation object to the file *self.Name+'.rlx'*.

SetMonoOrd(*ordr*)

Changes the default monomial order to *ord* which mustbe among *lex*, *grlex*, *grevlex*, *ilex*, *igrlex*, *igrevlex*.

SetNumCores(*num*)

Sets the maximum number of workers which cannot be bigger than number of available cores.

SetObjective(*obj*)

Takes the objective function *obj* as an algebraic combination of the generating symbolic functions, replace the symbolic functions with corresponding auxiliary symbols and reduce them according to the given relations.

SetSDPSolver(*solver*)

Sets the default SDP solver. The followings are currently supported:

- CVXOPT
- DSDP
- SDPA
- CSDP

The selected solver must be installed otherwise it cannot be called. The default solver is *CVXOPT* which has an interface for Python. *DSDP* is called through the CVXOPT's interface. *SDPA* and *CSDP* are called independently.

State()

Returns the latest state of the object at last break and save.

classmethod from_problem(*optim_prob:* OptimizationProblem, *name='SDPRlx'*)

Creates an SDPRelaxations instance from an OptimizationProblem.

This method acts as an alternative constructor to bridge compatibility with problems defined using SemigroupAlgebra.

Parameters

- **optim_prob** (OptimizationProblem) – The optimization problem defined with SemigroupAlgebra.
- **name** (*str*) – A name for the relaxation instance.

Returns
An instance of SDPRelaxations.

getConstraint(*idx*)

Returns the constraint number *idx* of the problem after reduction modulo the relations, if given.

getMomentConstraint(*idx*)

Returns the moment constraint number *idx* of the problem after reduction modulo the relations, if given.

getObjective()

Returns the objective function of the problem after reduction modulo the relations, if given.

pInitSDP()

Initializes the semidefinite program (SDP), in parallel, whose solution is a lower bound for the minimum of the program.

sInitSDP()

Initializes the semidefinite program (SDP), in serial mode, whose solution is a lower bound for the minimum of the program.

class Irene.relaxations.**SDRelaxSol**(*X*, *symdict={}*, *err_tol=1e-05*)

Instances of this class carry information on the solution of the semidefinite relaxation associated to an optimization problem. It includes various pieces of information:

- `SDRelaxSol.TruncatedMmntSeq` a dictionary of resulted moments
- `SDRelaxSol.MomentMatrix` the resulted moment matrix
- `SDRelaxSol.Primal` the value of the SDP in primal form
- `SDRelaxSol.Dual` the value of the SDP in dual form
- `SDRelaxSol.RunTime` the run time of the sdp solver
- `SDRelaxSol.InitTime` the total time consumed for initialization of the sdp
- `SDRelaxSol.Solver` the name of sdp solver
- `SDRelaxSol.Status` final status of the sdp solver
- `SDRelaxSol.RelaxationOrd` order of relaxation
- `SDRelaxSol.Message` the message that maybe returned by the sdp solver
- `SDRelaxSol.ScipySolver` the scipy solver to extract solutions
- `SDRelaxSol.err_tol` the minimum value which is considered to be nonzero
- `SDRelaxSol.Support` the support of discrete measure resulted from `SDPRelaxation.Minimize()`
- `SDRelaxSol.Weights` corresponding weights for the Dirac measures

ExtractSolution(*mthd='LH'*, *card=0*)

Extract support of the solution measure from `SDPRelaxations`:

-`mthd` should be either 'LH' or 'Scipy', where 'LH' stands for 'Lasserre-Henrion' and 'Scipy' employs a Scipy solver to find points matching the moments,

-`card` restricts the number of points of the support.

ExtractSolutionLH(*card=0*)

Extract solutions based on Lasserre–Henrion's method.

ExtractSolutionScipy(*card=0*)

This method tries to extract the corresponding values for generators of the `SDPRelaxation` class. Number of points is the rank of the moment matrix which is computed numerically according to the size of its eigenvalues. Then the points are extracted as solutions of a system of polynomial equations using a *scipy* solver. The following solvers are currently acceptable by `scipy`:

- `hybr`,
- `lm` (default),
- `broyden1`,
- `broyden2`,
- `anderson`,
- `linearmixing`,
- `diagbroyden`,
- `excitingmixing`,
- `krylov`,
- `df-sane`.

NumericalRank()

Finds the rank of the moment matrix based on the size of its eigenvalues. It considers those with absolute value less than `self.err_tol` to be zero.

Pivot(*arr*)

Get the leading term of each column.

SetScipySolver(*solver*)

Sets the `scipy.optimize.root` solver to *solver*.

StblRedEch(*A*)

Compute the stabilized row reduced echelon form.

Term2Mmnt(*trm*, *rnk*, *X*)

Converts a moment object into an algebraic equation.

class `Irene.sdp.`**sdp**(*solver='cvxopt'*, *solver_path=None*)

This is the class which intends to solve semidefinite programs in primal format:

$$\begin{cases} \min & \sum_{i=1}^{m} b_i x_i \\ \text{subject to} & \\ & \sum_{i=1}^{m} A_{ij} x_i - C_j \succeq 0 \quad j = 1, \ldots, k. \end{cases}$$

For the argument *solver* following sdp solvers are supported (if they are installed):

- *CVXOPT*,
- *CSDP*,
- *SDPA*,
- *DSDP*.

AddConstantBlock(*C*)

C must be a list of `numpy` matrices that represent C_j for each *j*. This method sets the value for $C = [C_1, \ldots, C_k]$.

AddConstraintBlock(*A*)

This takes a list of square matrices which corresponds to coefficient of x_i. Simply, $A_i = [A_{i1}, \ldots, A_{ik}]$. Note that the ith call of `AddConstraintBlock` fills the blocks associated with i^{th} variable x_i.

CvxOpt()

This calls *CVXOPT* and *DSDP* to solve the initiated semidefinite program.

Option(*param*, *val*)

Sets the *param* option of the solver to *val* if the solver accepts such an option. The following options are supported by solvers:

- `CVXOPT`:
  - `show_progress`: `True` or `False`, turns the output to the screen on or off (default: `True`);
  - `maxiters`: maximum number of iterations (default: 100);
  - `abstol`: absolute accuracy (default: 1e-7);
  - `reltol`: relative accuracy (default: 1e-6);
  - `feastol`: tolerance for feasibility conditions (default: 1e-7);
  - `refinement`: number of iterative refinement steps when solving KKT equations (default: 0 if the problem has no second-order cone or matrix inequality constraints; 1 otherwise).
- `SDPA`:
  - `maxIteration`: Maximum number of iterations. The SDPA stops when the iteration exceeds `maxIteration`;
  - `epsilonStar`, `epsilonDash`: The accuracy of an approximate optimal solution of the SDP;
  - `lambdaStar`: This parameter determines an initial point;
  - `omegaStar`: This parameter determines the region in which the SDPA searches an optimal solution;
  - `lowerBound`: Lower bound of the minimum objective value of the primal problem;
  - `upperBound`: Upper bound of the maximum objective value of the dual problem;
  - `betaStar`: Parameter controlling the search direction when current state is feasible;
  - `betaBar`: Parameter controlling the search direction when current state is infeasible;
  - `gammaStar`: Reduction factor for the primal and dual step lengths; 0.0 < `gammaStar` < 1.0.

SetObjective(*b*)

Takes the coefficients of the objective function.

static VEC(*M*)

Converts the matrix M into a column vector acceptable by *CVXOPT*.

csdp()

Calls *SDPA* to solve the initiated semidefinite program.

static parse_solution_matrix(*iterator*)

Parses and returns the matrices and vectors found by *SDPA* solver. This was taken from *ncpol2sdpa* and customized for *Irene*.

read_csdp_out(*filename*, *txt*)

Takes a file name and a string that are the outputs of *CSDP* as a file and command line outputs of the solver and extracts the required information.

read_sdpa_out(*filename*)

Extracts information from *SDPA*'s output file *filename*. This was taken from *ncpol2sdpa* and customized for *Irene*.

sdpa()

Calls *SDPA* to solve the initiated semidefinite program.

sdpa_param()

Produces sdpa.param file from `SolverOptions`.

solve()

Solves the initiated semidefinite program according to the requested solver.

write_sdpa_dat(*filename*)

Writes the semidefinite program in the file *filename* with dense SDPA format.

write_sdpa_dat_sparse(*filename*)

Writes the semidefinite program in the file *filename* with sparse SDPA format. Uses sparse iteration over non-zero entries to avoid dense nested loops.

class Irene.program.**OptimizationProblem**(*sga:* SemigroupAlgebra | *None* = *None*, *relations:* *list*[SemigroupAlgebraElement] | *None* = *None*)

This class represents an optimization problem.

sga

The semigroup algebra of the optimization problem.

Type
SemigroupAlgebra

relations

The relations of the optimization problem.

Type
list[*SemigroupAlgebraElement*]

semigroup

The semigroup of the optimization problem.

Type
CommutativeSemigroup

objective

The objective function of the optimization problem.

Type
SemigroupAlgebraElement

constraints

The constraints of the optimization problem.

Type
list[*SemigroupAlgebraElement*]

objective_degree

The degree of the objective function.

Type
int

objective_half_degree

The half degree of the objective function.

Type
int

constraints_degree

The degrees of the constraints.

Type
list[int]

constraints_half_degree

The half degrees of the constraints.

Type
list[int]

objective_trms_with_positive_coefficient

The terms of the objective function with positive coefficients.

Type
list[*SemigroupAlgebraElement*]

objective_trms_with_negative_coefficient

The terms of the objective function with negative coefficients.

Type
list[*SemigroupAlgebraElement*]

objective_terms_with_even_exponent

The terms of the objective function with even exponents.

Type
list[*SemigroupAlgebraElement*]

objective_terms_with_odd_exponent

The terms of the objective function with odd exponents.

Type
list[*SemigroupAlgebraElement*]

constraint_trms_with_positive_coefficient

The terms of the constraints with positive coefficients.

Type
list[*SemigroupAlgebraElement*]

constraint_trms_with_negative_coefficient

The terms of the constraints with negative coefficients.

Type
list[*SemigroupAlgebraElement*]

constraint_terms_with_even_exponent

The terms of the constraints with even exponents.

Type
list[*SemigroupAlgebraElement*]

constraint_terms_with_odd_exponent

The terms of the constraints with odd exponents.

Type
list[*SemigroupAlgebraElement*]

total_degree

The total degree of the optimization problem.

Type
int

add_constraints(*const: list[SemigroupAlgebraElement]*) → None

Adds polynomial inequality or equality constraints to the optimization problem.

This method extends the optimization problem with additional constraints, typically representing feasibility conditions like bounds or domain restrictions. Each constraint is analyzed for its degree to assist in later relaxation construction.

Place in code structure: Complements set_objective() to fully define the constrained optimization problem. Called during problem setup before relaxation methods.

Parameters
const (*list[SemigroupAlgebraElement]*) – List of polynomial constraint expressions, each represented as a SemigroupAlgebraElement.

Returns
None

Side effects:

- Appends each constraint to self.constraints
- Updates self.semigroup if any constraint uses a different semigroup
- Computes and stores degree and half-degree for each constraint

analyse_program() → None

Performs structural analysis of the optimization problem's polynomial terms.

This method categorizes all terms from the objective function and constraints based on two key properties: (1) sign of coefficients, and (2) parity of exponents. This analysis is essential for specialized polynomial optimization techniques like SONC (sums of non-negative circuit polynomials) and conditional relaxations.

Place in code structure: Called after problem setup to preprocess the polynomial structure. Populates classification lists used by relaxation algorithms (relaxations.py) and decomposition methods (sonc.py).

Returns
None

Side effects:
Populates the following attributes: - objective_trms_with_positive/negative_coefficient: Objective terms by sign - constraint_trms_with_positive/negative_coefficient: Constraint terms by sign - objective_terms_with_even/odd_exponent: Objective terms by exponent parity - constraint_terms_with_even/odd_exponent: Constraint terms by exponent parity

convex_combination(*point: Sequence[float]*) → ndarray | None

Finds the representation of a point as a convex combination of Newton polytope vertices.

This method formulates and solves a linear program to express a point as a convex combination of the polytope vertices, enforcing both non-negativity of coefficients and that they sum to 1. This is more restrictive than linear_combination() but guarantees the point lies within the convex hull.

Place in code structure: Advanced geometric utility for Newton polytope analysis. Uses scipy.optimize to solve the constrained optimization problem. Results can guide sparse relaxation strategies by identifying how to express new terms using existing ones.

Parameters

point – A numeric sequence representing coordinates in the exponent space that should be expressed as a convex combination of self.vertices.

Returns

Array of non-negative coefficients summing to 1 that express the point
as a convex combination of vertices, or None if no such combination exists (i.e., point is outside the Newton polytope).

Return type
numpy.ndarray

delta(*xprsn: SemigroupAlgebraElement, deg: int*) → dict[str, set]

Identifies problematic terms in polynomial expressions for relaxation construction.

The delta function categorizes terms that cannot be directly represented in a sum-of-squares (SOS) form: terms with odd exponents or negative coefficients. These terms are grouped by whether their total degree equals the target degree ('=d') or is less than it ('<d'), which affects how they're handled in relaxations.

Place in code structure: Core analysis method used by relaxation algorithms (relaxations.py) to identify which terms need special treatment through auxiliary variables or alternative decomposition strategies.

Parameters

- **xprsn** (SemigroupAlgebraElement) – The polynomial expression to analyze.
- **deg** (*int*) – The target degree for comparison.

Returns

Dictionary with two keys:
'=d': Set of monomials with degree exactly equal to deg '<d': Set of monomials with degree less than deg Both contain only non-square or negative coefficient terms.

Return type
dict[str, set[*SemigroupAlgebraElement*]]

delta_vertex(*xprsn: SemigroupAlgebraElement, vertices: list*) → None

Computes delta relative to Newton polytope vertices (currently unimplemented).

This method is intended to analyze terms based on their relationship to the vertices of the Newton polytope, likely for geometric programming or sparsity exploitation.

Place in code structure: Placeholder for advanced geometric analysis that would work with the newton() and related polytope methods for exploiting problem structure.

Parameters

- **xprsn** (SemigroupAlgebraElement) – The expression to analyze.
- **vertices** (*list*) – List of vertices from the Newton polytope.

Returns
None (not yet implemented)

static has_symbol(*symb: str*, *mono:* SemigroupAlgebraElement) → tuple[bool, int]

Searches for a specific variable (symbol) in a monomial and returns its position.

This method determines whether a given variable name appears in a monomial's factorization, which is useful for variable-specific operations like substitution, elimination, or targeted relaxation strategies.

Place in code structure: Static utility method for monomial inspection. Used by various algorithms that need to check variable presence or extract variable-specific information from polynomial terms.

Parameters

- **symb** (*str*) – The name of the variable/symbol to search for (e.g., 'x', 'y').
- **mono** (SemigroupAlgebraElement) – The monomial to search within.

Returns

A tuple where the first element is True if the symbol is found,
False otherwise. The second element is the index of the symbol in the monomial's array form if found, or -1 if not found.

Return type
tuple[bool, int]

in_newton(*point: Sequence[float]*) → bool

Tests whether a point lies inside the Newton polytope.

This method uses Delaunay triangulation to efficiently determine if a given point (typically representing a monomial's exponents) is within the convex hull of the problem's support. This is useful for determining which monomials are representable as convex combinations of existing terms.

Place in code structure: Geometric query method that depends on newton() being called first. Used in sparsity analysis and to determine valid monomial spaces for relaxations.

Parameters
point – A numeric sequence (tuple, list, or array) representing exponent coordinates to test for inclusion in the Newton polytope.

Returns

True if the point is inside or on the boundary of the Newton polytope,
False otherwise.

Return type
bool

Raises
ValueError – If vertices are empty or if Delaunay triangulation fails due to degenerate geometry.

linear_combination(*point: Sequence[float]*) → ndarray

Expresses a point as a linear combination of Newton polytope vertices.

This method solves a linear system to find coefficients that express the given point as a linear combination of polytope vertices. Note that unlike convex_combination(), this method doesn't enforce non-negativity or that coefficients sum to 1, so it's suitable for affine (not necessarily convex) combinations.

Place in code structure: Geometric utility for expressing points in terms of vertices. Complements convex_combination() but with less restrictive constraints. Used when affine relationships are needed rather than strictly convex ones.

Parameters
point – A numeric sequence representing coordinates to express in terms of vertices.

Returns

Array of coefficients for the linear combination. Note that these
coefficients may be negative or sum to values other than 1.

Return type
numpy.ndarray

> **Note**
>
> Excludes the origin vertex [0, 0, . . .] if present to avoid singularity issues.

mono2ord_tuple(*mono: Any*) → tuple[int, ...]

Converts a monomial from semigroup representation to a tuple of exponents.

This method transforms a symbolic monomial into a numeric tuple where each position corresponds to a generator in the semigroup, containing that generator's exponent. This standardized format is essential for numerical computations like convex hull calculations and geometric analysis.

Place in code structure: Conversion utility used by newton() and related geometric methods to transform symbolic polynomial data into numeric arrays suitable for scipy geometric algorithms.

Parameters
mono (SemigroupAlgebraElement) – The monomial to convert. Can also be a scalar (int or float) representing a constant term.

Returns

A tuple of integers representing the exponents of each generator in order.
For constants, returns a tuple of zeros.

Return type
tuple

newton() → None

Computes the Newton polytope of the optimization problem.

The Newton polytope is the convex hull of all exponent vectors appearing in the objective function and constraints. This geometric object encodes the sparsity structure of the problem and can be exploited to construct more efficient relaxations.

Place in code structure: Geometric analysis method that should be called after problem setup. Its results (stored in newton_polytope and vertices) are used by in_newton(), convex_combination(), and potentially specialized relaxation strategies.

Returns
None

Side effects:

- Collects all exponent tuples from objective and constraints
- Computes and stores the convex hull in self.newton_polytope
- Extracts and stores sorted vertex list in self.vertices
- Raises ValueError if polytope is degenerate (insufficient points or dimension mismatch)

static omega(*xprsn:* SemigroupAlgebraElement, *deg:* *int*) → list[tuple[float, Any]]

Extracts terms from a polynomial that don't match a specific univariate degree pattern.

The omega function filters out univariate monomials of exactly the specified degree, keeping all multivariate terms, constant terms, and univariate terms of other degrees. This is used in conditional optimization where certain degree-specific terms need special treatment.

Place in code structure: Static helper method used by delta() to identify terms that require special handling in relaxation construction, particularly for conditional constraints and geometric programming.

Parameters

- **xprsn** (SemigroupAlgebraElement) – The polynomial expression to filter.
- **deg** (*int*) – The specific univariate degree to exclude.

Returns

List of (coefficient, monomial) tuples
representing terms that don't match the degree criterion.

Return type
list[tuple[int, *SemigroupAlgebraElement*]]

program_degree() → int

Computes the overall degree of the optimization problem.

This method determines the maximum degree among all polynomials (objective and constraints) in the problem, rounded up to the nearest even number. This degree is critical for determining the size of sum-of-squares (SOS) relaxations.

Place in code structure: Utility method used during relaxation construction to determine the appropriate degree for SOS decompositions and moment matrices.

Returns

The maximum degree among all polynomials, adjusted to be even (adds 1 if odd).
This ensures compatibility with SOS relaxations which require even degrees.

Return type
int

set_objective(*obj:* SemigroupAlgebraElement) → None

Sets the objective function for the optimization problem.

This method is central to defining the optimization problem. It assigns the objective function that will be minimized or maximized, and automatically updates the semigroup and computes the degree information needed for polynomial optimization.

Place in code structure: Core setup method that must be called before relaxation generation. Works with add_constraints() to fully specify the optimization problem.

Parameters
obj (SemigroupAlgebraElement) – A polynomial expression to be optimized, represented as an element in a semigroup algebra.

Returns
None

Side effects:

- Updates self.objective with the provided expression
- Updates self.semigroup if different from current

- Computes and stores objective_degree (total degree of the leading monomial)
- Computes and stores objective_half_degree (ceiling of degree/2)

static square_exponent(*xpnt: Any*) → bool

Determines if a monomial has only even exponents (is a perfect square).

This method checks whether all variables in a monomial appear with even powers, meaning the monomial can be expressed as a square of another monomial. This property is fundamental for identifying terms that are inherently non-negative and for constructing sum-of-squares decompositions.

Place in code structure: Static utility method used throughout analysis and relaxation construction. Called by analyse_program(), delta(), and other methods that need to identify square monomials for optimization techniques.

Parameters
xpnt (SemigroupAlgebraElement) – A monomial whose exponents are to be checked.

Returns

True if all exponents in the monomial are even (divisible by 2),
False if any exponent is odd.

Return type
bool

to_sympy(*expr:* SemigroupAlgebraElement, *sym_map: dict[str, Symbol]*)

Converts a SemigroupAlgebraElement polynomial to a SymPy symbolic expression.

This method provides interoperability with the SymPy computer algebra system, enabling use of SymPy's extensive symbolic manipulation capabilities (differentiation, integration, simplification, etc.) on polynomials defined in the semigroup algebra framework.

Place in code structure: Interface method for exporting to SymPy. Useful for visualization, symbolic analysis, or interfacing with other tools that expect SymPy expressions. Complements the internal semigroup algebra representation.

Parameters

- **expr** (SemigroupAlgebraElement) – The polynomial expression to convert.
- **sym_map** (*dict*) – Dictionary mapping generator name strings to SymPy Symbol objects, e.g., {'x': Symbol('x'), 'y': Symbol('y')}.

Returns

A SymPy expression algebraically equivalent to the input, using the
symbols provided in sym_map.

Return type
sympy.Expr

tuple2mono(*xpnt: Sequence[int]*)

Converts a tuple of exponents back to a monomial in semigroup representation.

This method performs the inverse operation of mono2ord_tuple, constructing a symbolic monomial from numeric exponents. Each position in the input tuple corresponds to the power of the respective generator in the semigroup.

Place in code structure: Inverse conversion utility that works with mono2ord_tuple to enable round-trip conversions between symbolic and numeric representations. Used when translating geometric analysis results back to polynomial terms.

Parameters
xpnt – A sequence (tuple or list) of integers representing exponents for each generator in the semigroup, in the same order as semigroup.generators.

Returns

The monomial product of generators raised to their
corresponding powers from xpnt.

Return type
SemigroupAlgebraElement

A module for working with commutative semigroups and their differential algebras.

This module provides classes and functions for working with commutative semigroups and their differential algebras.

A semigroup is a set S together with an associative binary operation on S. A commutative semigroup is a semigroup in which the binary operation is commutative.

A semigroup algebra is a vector space over a field with a basis consisting of the elements of a semigroup equipped with multiplication compatible with the semigroup's operation.

This module provides the following classes:

- CommutativeSemigroup: A class representing a commutative semigroup.
- AtomicSGElement: A class representing an atomic element of a semigroup algebra.
- SemigroupAlgebraElement: A class representing an element of a semigroup algebra.
- SemigroupAlgebra: A class representing a semigroup algebra.

This module also provides the following functions:

- _degree: Computes the degree of a given expression.
- diff: Computes the derivative of an expression in a semigroup algebra.

class Irene.grouprings.**AtomicSGElement**(*semigroup:* CommutativeSemigroup, *element: str*)

An atomic element of a semigroup algebra.

An atomic element is an element of the semigroup algebra that is not a sum of other elements.

semigroup

The semigroup of the element.

Type
CommutativeSemigroup

symbol

The symbol of the element.

Type
str

content

The content of the element.

Type
list

LC() → float

Returns the leading coefficient of the element.

The leading coefficient of an element is the coefficient of the term with the highest degree.

Returns
The leading coefficient of the element.

Return type
float

LM() → Expr

Returns the leading monomial of the element.

The leading monomial of an element is the monomial with the highest degree.

Returns
The leading monomial of the element.

Return type
sympy.Expr

LT() → *SemigroupAlgebraElement*

Returns the leading term of the element.

The leading term of an element is the term with the highest degree.

Returns
The leading term of the element.

Return type
SemigroupAlgebraElement

divide(*fs: list*) → tuple[list, *SemigroupAlgebraElement*]

Divides the element by a list of elements or expressions.

Parameters
fs (*list*) – The list of elements or expressions to divide by.

Returns
A tuple containing the quotient and remainder of the division.

Return type
tuple

lt_divisible_by(*expr: Any*) → bool

Checks if the leading term of the element is divisible by the leading term of another element or expression.

Parameters
expr (*int*, *float*, AtomicSGElement, SemigroupAlgebraElement) – The element or expression to check divisibility by.

Returns
True if the leading term of the element is divisible by the leading term of the other element or expression, False otherwise.

Return type
bool

class Irene.grouprings.**CommutativeSemigroup**(*gens: list*, *is_semigroup: bool* = *True*, *is_abelian: bool* = *True*)

A class representing a commutative semigroup.

A semigroup is a set S together with an associative binary operation on S. A commutative semigroup is a semigroup in which the binary operation is commutative.

gens

The generators of the semigroup.

Type
list

is_semigroup

True if the semigroup is a semigroup, False otherwise.

Type
bool

is_abelian

True if the semigroup is abelian, False otherwise.

Type
bool

rels

The relations of the semigroup.

Type
list

aux_rels

The auxiliary relations of the semigroup.

Type
list

inverses

The inverses of the generators of the semigroup.

Type
dict

max_deg

The maximum degree of the relations of the semigroup.

Type
int

edges

The edges of the lattice of the semigroup.

Type
list

vertices

The vertices of the lattice of the semigroup.

Type
dict

symbols

The symbols of the semigroup.

Type
list

add_relations(*rels: list*)

Adds relations to the semigroup.

Parameters
rels (*list*) – The relations to add to the semigroup.

degree(*expr*) → int

Computes the degree of an expression in the semigroup.

The degree of an expression is the sum of the absolute values of the exponents of its terms.

Parameters
expr – The expression whose degree is to be computed.

Returns
The degree of the expression.

Return type
int

element_sub_lattice(*elm: Expr*, *ex: set = None*) → set

Computes the sublattice of the lattice of the semigroup generated by an element.

Parameters

- **elm** (*sympy.Expr*) – The element of the semigroup to generate the sublattice from.
- **ex** (*set*) – The set of elements to exclude from the sublattice.

Returns
The sublattice of the lattice of the semigroup generated by the element.

Return type
set

lattice_edges(*degree: int*) → None

Computes the edges of the lattice of the semigroup.

Parameters
degree (*int*) – The degree of the lattice.

lattice_vertices() → None

Computes the vertices of the lattice of the semigroup.

The vertices of the lattice are the elements of the semigroup that are generated by the edges of the lattice.

positive_exp(*expr: Expr*) → bool

Checks if an expression in the semigroup has only positive exponents.

Parameters
expr (*sympy.Expr*) – The expression to check.

Returns
True if the expression has only positive exponents, False otherwise.

Return type
bool

class Irene.grouprings.**SemigroupAlgebra**(*semigroup: CommutativeSemigroup*)

A semigroup algebra.

A semigroup algebra is a vector space over a field with a basis consisting of the elements of a semigroup.

gens

The generators of the semigroup.

Type
list

semigroup

The semigroup of the algebra.

Type
CommutativeSemigroup

derivatives

A list of dictionaries mapping the elements of the semigroup to their derivatives.

Type
list

add_derivative(*base_map: dict*) → None

Adds a derivative to the algebra.

Parameters
base_map (*dict*) – A dictionary mapping the elements of the semigroup to their derivatives.

derivative(*expr: Any*, *idx: int*)

Computes the derivative of an expression in the algebra.

Parameters

- **expr** (SemigroupAlgebraElement, AtomicSGElement, *int*, *float*) – The expression to differentiate.
- **idx** (*int*) – The index of the derivative to use.

Returns
The derivative of the expression.

Return type
SemigroupAlgebraElement

diff(*expr: Any*, *base_map: dict*)

Computes the derivative of an expression in the algebra.

Parameters

- **expr** (SemigroupAlgebraElement, AtomicSGElement, *int*, *float*) – The expression to differentiate.
- **base_map** (*dict*) – A dictionary mapping the elements of the semigroup to their derivatives.

Returns
The derivative of the expression.

Return type
SemigroupAlgebraElement

class Irene.grouprings.**SemigroupAlgebraElement**(*terms: list*, *semigroup:* CommutativeSemigroup)

An element of a semigroup algebra.

A semigroup algebra is a vector space over a field with a basis consisting of the elements of a semigroup.

content

The content of the element.

Type
list

semigroup

The semigroup of the element.

Type
CommutativeSemigroup

LC() → float

Returns the leading coefficient of the element.

The leading coefficient of an element is the coefficient of the term with the highest degree.

Returns
The leading coefficient of the element.

Return type
float

LM() → Expr

Returns the leading monomial of the element.

The leading monomial of an element is the monomial with the highest degree.

Returns
The leading monomial of the element.

Return type
sympy.Expr

LT() → *SemigroupAlgebraElement*

Returns the leading term of the element.

The leading term of an element is the term with the highest degree.

Returns
The leading term of the element.

Return type
SemigroupAlgebraElement

divide(*fs: list*) → tuple[list, *SemigroupAlgebraElement*]

Divides the element by a list of elements or expressions.

Parameters
fs (*list*) – The list of elements or expressions to divide by.

Returns
A tuple containing the quotient and remainder of the division.

Return type
tuple

lt_divisible_by(*expr: Any*) → bool

Checks if the leading term of the element is divisible by the leading term of another element or expression.

Parameters
expr (*int*, *float*, AtomicSGElement, SemigroupAlgebraElement) – The element or expression to check divisibility by.

Returns
True if the leading term of the element is divisible by the leading term of the other element or expression, False otherwise.

Return type
bool

This module provides a framework for polynomial optimization using the techniques introduced by Ghasemi, Lasserre, and Marshall, using Geometric Programming.

class `Irene.geometric.GPRelaxations`(*prog:* OptimizationProblem, ***kwargs*)

This class aims to provide a framework for polynomial optimization using the techniques introduced by Ghasemi, Lasserre, and Marshall, using Geometric Programming.

auto_transform_matrix() → ndarray

Compute the automatic transformation matrix H.

Returns
The transformation matrix H.

static compare_diags(*vec: list[float]*) → tuple[int, list[float]]

Compare two vectors by the number of non-zero elements.

Parameters
vec – The first vector.

Returns
The number of non-zero elements in the vector.

h_plus(*xprsn: Any*, *idn: Any = None*) → float

Compute the h_plus function for a given expression.

Parameters

- **xprsn** – The expression to evaluate.
- **idn** – The identity element of the semigroup.

Returns
The value of h_plus(xprsn).

solve() → float

Form the geometric program relaxation.

Returns
The optimal value of the relaxation.

transform_program() → None

Transform the program using the transformation matrix H.

class `Irene.sonc.SONCRelaxations`(*prog:* OptimizationProblem, ***kwargs*)

Framework for constrained SONC relaxations formulated as geometric programs.

solve(*verbosity: int | None = None*) → float

Build and solve the constrained SONC geometric program.

CHAPTER

THIRTEEN

PYPROXIMATION (LEAN)

This is a brief documentation for using *pyProximation*. pyProximation provides tools for function approximation in Hilbert spaces. It uses orthogonal systems of functions built via the Gram-Schmidt procedure over measures on arbitrary domains.

13.1 Requirements and dependencies

Since this is a python package, so clearly one needs have python available. pyProximation relies on the following packages:

- **for numerical calculation:**
  - numpy,
  - scipy ,
- **for symbolic computation:**
  - sympy

13.2 Download

pyProximation can be obtained from https://github.com/mghasemi/pyProximation.

13.3 Installation

To install *pyProximation*, run the following in terminal:

```
sudo python setup.py install
```

13.3.1 Documentation

The documentation of *pyProximation* is prepaqred via sphinx.

13.4 License

pyProximation is distributed under MIT license:

13.4.1 MIT License

Copyright (c) 2016 Mehdi Ghasemi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

CHAPTER

FOURTEEN

MEASURES

14.1 Density function

The *pyProximation* implements two different scenarios for measure spaces:

1. Continuous case, where support of the measure is given as a compact subspace (box) of $\mathbb{R}^n$, and
2. Discrete case, where a finite set of points and their weights are given.

14.1.1 Continuous measure spaces

For the continuous case, generally assume that the support of the measure is a product of closed interval, i.e., $\prod_{i=1}^n [a_i, b_i]$, where for each i, $a_i < b_i$. Such a set can be defined as a list of ordered pairs of real numbers as `[(a1, b1), (a2, b2), ..., (an, bn)]`. Moreover, when we speak about a subset of the support, we always refer to a box, defined as a list of 2-tuples.

In this case, the measure is given implicitly as a density function $w(x)$. So the measure of a set S is given by

$$\mu(S) = \int_S w(x)dx.$$

For example, the following code defines the Lebesgue measure on $[-1, 1] \times [-1, 1]$ and finds the measure of the set $[0, 1] \times [-1, 0]$:

```
# import the Measure class
from pyProximation import Measure
# define the support of the measure
D = [(-1, 1), (-1, 1)]
# define the measure with the constant density 1
M = Measure(D, 1)
# define a set called S
S = [(0, 1), (-1, 0)]
# find the measure of the set S
print M.measure(S)
```

14.1.2 Discrete measure spaces

In this case, the measure is basically a convex combination of Dirac measure. Given a set $X = \{x_1, \ldots, x_n\}$ and corresponding non-negative weights $w_1, \ldots, w_n$, one defines a measure as $\mu = \sum_{i=1}^n w_i \delta_{x_i}$. Then the measure of a subset $S = \{x_{i_1}, \ldots, x_{i_k}\}$ of X is given by

$$\mu(S) = \int_S d\mu = \sum_{j=1}^k w_{i_j}$$

The following is a sample code for discrete case:

```
# import the Measure class
from pyProximation import Measure
# define the support and density
D = {'x1':1, 'x2':.5, 'x3':1.1, 'x4':.6}
# define the measure
M = Measure(D)
# define a set called S
S = ['x2', 'x3']
# find the measure of the set S
print M.measure(S)
```

14.2 Integrals

Suppose that a measure space (X, μ) and a measurable function f on X are given. The method `integral` computes $\int_X f d\mu$. If μ is discrete, then f can be a dictionary with keys as points of domain and values as evaluation at each point. Otherwise, f is simply a numerical function:

```
from pyProximation import Measure
from numpy import sqrt
# define the density function
w = lambda x:1./sqrt(1.-x**2)
# define the support
D = [(-1, 1)]
# initiate the measure space
M = Measure(D, w)
# set f(x) = x^2
f = lambda x: x**2
# integrate f(x) w.r.t. w(x)
print M.integral(f)
```

Or in two dimensions:

```
from pyProximation import Measure
from numpy import sqrt
# define the density function
w = lambda x, y:y**2/sqrt(1.-x**2)
# define the support
D = [(-1, 1), (-1, 1)]
# initiate the measure space
M = Measure(D, w)
# set f(x, y) = x^2 + y
f = lambda x, y: x**2 + y
# integrate f(x, y) w.r.t. w(x, y)
print M.integral(f)
```

14.3 *p*-norms

Given a measure space (X, μ) and a measurable function f, the p-norm of f, for a positive p is defined as:

$$\|f\|_p = \left(\int_X |f|^p d\mu\right)^{1/p}.$$

The method `norm(p, f)` calculates the above quantity.

14.4 Drawing samples

Suppose that (X, μ) is a measure space and one wishes to draw a sample of size n from X according to the distribution μ. This can be done by the method `sample(n)` which returns a list of n random points from the support, according to μ.

CHAPTER

FIFTEEN

HILBERT SPACES

15.1 Orthonormal system of functions

Let X be a topological space and μ be a finite Borel measure on X. The bilinear function $\langle \cdot, \cdot \rangle$ defined on $L^2(X, \mu)$ as $\langle f, g \rangle = \int_X fg d\mu$ is an inner product which turns $L^2(X, \mu)$ into a Hilbert space.

Let us denote the family of all continuous real valued functions on a non-empty compact space X by $\mathrm{C}(X)$. Suppose that among elements of $\mathrm{C}(X)$, a subfamily A of functions are of particular interest. Suppose that A is a subalgebra of $\mathrm{C}(X)$ containing constants. We say that an element $f \in \mathrm{C}(X)$ can be approximated by elements of A, if for every $\epsilon > 0$, there exists $p \in A$ such that $|f(x) - p(x)| < \epsilon$ for every $x \in X$. The following classical results guarantees when every $f \in \mathrm{C}(X)$ can be approximated by elements of A.

Note

Stone-Weierstrass:

Every element of $\mathrm{C}(X)$ can be approximated by elements of A if and only if for every $x \neq y \in X$, there exists $p \in A$ such that $p(x) \neq p(y)$.

Despite the strong and important implications of the Stone-Weierstrass theorem, it leaves every computational details out and does not give an specific algorithm to produce an estimator for f with elements of A, given an error tolerance ϵ, and the search for a such begins.

Define $\|f\|_\infty$ (the $\sup$ norm of f) of a given function $f \in \mathrm{C}(X)$ by

$$\|f\|_\infty = \sup_{x \in X} |f(x)|,$$

Then the above argument can be read as: For every $f \in \mathrm{C}(X)$ and every $\epsilon > 0$, there exists $p \in A$ such that $\|f - p\|_\infty < \epsilon$.

Let $(V, \langle \cdot, \cdot \rangle)$ be an inner product space with $\|v\|_2 = \langle v, v \rangle^{\frac{1}{2}}$. A basis $\{v_\alpha\}_{\alpha \in I}$ is called an orthonormal basis for V if $\langle v_\alpha, v_\beta \rangle = \delta_{\alpha\beta}$, where $\delta_{\alpha\beta} = 1$ if and only if $\alpha = \beta$ and is equal to *0* otherwise. Every given set of linearly independent vectors can be turned into a set of orthonormal vectors that spans the same sub vector space as the original. The following well-known result gives an algorithm for producing such orthonormal from a set of linearly independent vectors:

Note

Gram–Schmidt

Let $(V, \langle \cdot, \cdot \rangle)$ be an inner product space. Suppose $\{v_i\}_{i=1}^n$ is a set of linearly independent vectors in V. Let

$$u_1 := \frac{v_1}{\|v_1\|_2}$$

and (inductively) let

$$w_k := v_k - \sum_{i=1}^{k-1} \langle v_k, u_i \rangle u_i \text{ and } u_k := \frac{w_k}{\|w_k\|_2}.$$

Then $\{u_i\}_{i=1}^n$ is an orthonormal collection, and for each *k*,

$$span\{u_1, u_2, \cdots, u_k\} = span\{v_1, v_2, \cdots, v_k\}.$$

Note that in the above note, we can even assume that $n = \infty$.

Let $B = \{v_1, v_2, \dots\}$ be an ordered basis for $(V, \langle \cdot, \cdot \rangle)$. For any given vector $w \in V$ and any initial segment of *B*, say $B_n = \{v_1, \dots, v_n\}$, there exists a unique $v \in \text{span}(B_n)$ such that $\|w - v\|_2$ is the minimum:

Note

Let $w \in V$ and *B* a finite orthonormal set of vectors (not necessarily a basis). Then for $v = \sum_{u \in B} \langle u, w \rangle u$

$$\|w - v\|_2 = \min_{z \in \text{span}(B)} \|w - z\|_2.$$

Now, let μ be a finite measure on *X* and for $f, g \in \mathrm{C}(X)$ define $\langle f, g \rangle = \int_X fg d\mu$. This defines an inner product on the space of functions. The norm induced by the inner product is denoted by $\| \cdot \|_2$. It is evident that

$$\|f\|_2 \leq \|f\|_\infty \mu(X), \ \forall f \in \mathrm{C}(X),$$

which implies that any good approximation in $\| \cdot \|_\infty$ gives a good $\| \cdot \|_2$-approximation. But generally, our interest is the other way around. Employing Gram-Schmidt procedure, we can find $\| \cdot \|_2$ within any desired accuracy, but this does not guarantee a good $\| \cdot \|_\infty$-approximation. The situation is favorable in finite dimensional case. Take $B = \{p_1, \dots, p_n\} \subset \mathrm{C}(X)$ and $f \in \mathrm{C}(X)$, then there exists $K_f > 0$ such that for every $g \in \text{span}(B \cup \{f\})$,

$$K_f \|g\|_\infty \leq \|g\|_{2 \leq} \|g\|_\infty \mu(X).$$

Since *X* is assumed to be compact, $\mathrm{C}(X)$ is separable, i.e., $\mathrm{C}(X)$ admits a countable dimensional dense subvector space (e.g. polynomials for when *X* is a closed, bounded interval). Thus for every $f \in \mathrm{C}(X)$ and every $\epsilon > 0$ one can find a big enough finite *B*, such that the above inequality holds. In other words, good enough $\| \cdot \|_2$-approximations of f give good $\| \cdot \|_\infty$-approximations, as desired.

15.2 OrthSystem

Given a measure space, the `OrthSystem` class implements the described procedure, symbolically. Therefore, it relies on a symbolic environment. Currently, the supported symbolic environment is:

1. *sympy*

15.2.1 Legendre polynomials

Let $d\mu(x) = dx$, the regular Lebesgue measure on $[-1, 1]$ and $B = \{1, x, x^2, \dots, x^n\}$. The orthonormal polynomials constructed from *B* are called *Legendre* polynomials. The nth Legendre polynomial is denoted by $P_n(x)$.

The following code generates Legendre polynomials up to a given order:

```
# the symbolic package
from sympy import *
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 6
# define the measure
D = [(-1, 1)]
M = Measure(D, 1)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.PolyBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# print the result
print(S.OrthBase)
```

15.2.2 Chebyshev polynomials

Let $d\mu(x) = \frac{dx}{\sqrt{1-x^2}}$ on $[-1, 1]$ and B as in Legendre polynomials. The orthonormal polynomials associated to this setting are called *Chebyshev* polynomials and the nth one is denoted by $T_n(x)$.

The following code generates Chebyshev polynomials up to a given order:

```
# the symbolic package
from sympy import *
from numpy import sqrt
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 6
# define the measure
D = [(-1, 1)]
w = lambda x: 1./sqrt(1. - x**2)
M = Measure(D, w)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.PolyBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# print the result
print(S.OrthBase)
```

15.3 Approximation

Let (X, μ) be a compact Borel measure space and $\mathcal{O} = \{u_1, u_2, \dots\}$ an orthonormal basis of function whose span is dense in $L^2(X, \mu)$. Given a function $f \in L^2(X, \mu)$, then f can be approximated as

$$f = \lim_{n\to\infty} \sum_{i=1}^{n} \langle f, u_i \rangle u_i$$

`OrthSystem.Series` calculates the coefficients $\langle f, u_i \rangle$:

15.3.1 Truncated Fourier series

Let $d\mu(x) = dx$, the regular Lebesgue measure on $[c, c + 2l]$ and $B = \{1, \sin(\pi x), \cos(\pi x), \sin(2\pi x), \cos(2\pi x), \dots, \sin(n\pi x), \cos(n\pi x), \}$. The following code calculates the Fourier series approximation of $f(x) = \sin(x)e^x$:

```
from sympy import *
from numpy import sqrt
from pyProximation import Measure, OrthSystem
# the symbolic variable
x = Symbol('x')
# set a limit to the order
n = 4
# define the measure
D = [(-1, 1)]
w = lambda x: 1./sqrt(1. - x**2)
M = Measure(D, w)
S = OrthSystem([x], D, 'sympy')
# link the measure to S
S.SetMeasure(M)
# set B = {1, x, x^2, ..., x^n}
B = S.FourierBasis(n)
# link B to S
S.Basis(B)
# generate the orthonormal basis
S.FormBasis()
# number of elements in the basis
m = len(S.OrthBase)
# set f(x) = sin(x)e^x
f = sin(x)*exp(x)
# extract the coefficients
Coeffs = S.Series(f)
# form the approximation
f_app = sum([S.OrthBase[i]*Coeffs[i] for i in range(m)])
print(f_app)
```

15.3.2 Fitting Data

Suppose that a set of data points $(x_1, y_1), \dots, (x_m, y_m)$ are given, where $x_i \in \mathbb{R}^n$. To fit a curve or surface to data with minimum average square error, one can employ `OrthSystem` with an arbitrary system of functions to do so. The following example generates random data and fits two surfaces using polynomials and trigonometric functions:

```
from random import uniform
from sympy import Function, Symbol
from sympy.utilities.lambdify import implemented_function, lambdify
# import the module
from pyProximation import Measure, OrthSystem
x = Symbol('x')
y = Symbol('y')
# define dirac function symbolically and numerically
dirac = implemented_function(
    Function('dirac'), lambda a, b: 1 * (abs(a - b) < .000001))
# list of random points
points = [(uniform(0, 1), uniform(0, 1)) for _ in range(50)]
# container for the support of discrete measure
D = {}
# a symbolic function to represent the random points
g = 0
# Random values of the function
vals = []
for p in points:
    t = uniform(-.8, 1)
    vals.append(t)
    g += t * dirac(x, p[0]) * dirac(y, p[1])
    D[p] = 1
# The discrete measure supported at random points
M = Measure(D)
# Orthonormal systems of functions
S1 = OrthSystem([x, y], [(0, 1), (0, 1)])
S1.SetMeasure(M)

S2 = OrthSystem([x, y], [(0, 1), (0, 1)])
S2.SetMeasure(M)
# polynomial basis
B1 = S1.PolyBasis(4)
# trigonometric basis
B2 = S2.FourierBasis(3)
# link the basis to the orthogonal systems
S1.Basis(B1)
S2.Basis(B2)
# form the orthonormal basis
S1.FormBasis()
S2.FormBasis()
# calculate coefficients
cfs1 = S1.Series(g)
cfs2 = S2.Series(g)
# orthogonal approximation
aprx1 = sum([S1.OrthBase[i] * cfs1[i] for i in range(len(S1.OrthBase))])
aprx2 = sum([S2.OrthBase[i] * cfs2[i] for i in range(len(S2.OrthBase))])
# inspect the symbolic approximations
print(aprx1)
print(aprx2)
# compare the first sample numerically
f1 = lambdify((x, y), aprx1, 'numpy')
f2 = lambdify((x, y), aprx2, 'numpy')
```

(continues on next page)

(continued from previous page)

```
print(vals[0], f1(*points[0]), f2(*points[0]))
```

15.4 Rational Approximation

Let $f : X \to \mathbb{R}$ be a continuous function and $\{b_0, b_1, \dots\}$ an orthonormal system of functions over X with respect to a measure μ. Suppose that we want to approximate f with a function of the form $\frac{p(x)}{q(x)}$ where $p = \sum_{j=0}^{m} \alpha_j b_j$ and $q = 1 + \sum_{j=1}^{n} \beta_j b_j$. Let $r(x) = p(x) - q(x)f(x)$ be the residual. We can find the coefficients α_i, β_j such that $\|r\|_2$ be minimum.

Let $L(\alpha, \beta) = r(x) \cdot r(x)$. Then the solution satisfies the following equations:

$$\frac{\partial}{\partial \alpha_i} L = 0,$$
$$\frac{\partial}{\partial \beta_i} L = 0,$$

which is a system of linear equations. This is implemented in `rational.RationalApprox.RatLSQ`:

```
from sympy import Symbol, cos, exp, pi, sin, sinh
from pyProximation import OrthSystem, RationalApprox

x = Symbol('x')
f = sinh(x) * cos(3 * x) + sin(3 * x) * exp(x)
D = [(-pi, pi)]
S = OrthSystem([x], D)
B = S.PolyBasis(5)

# link B to S
S.Basis(B)

# generate the orthonormal basis
S.FormBasis()

# extract the coefficients of approximations
Coeffs = S.Series(f)

# form the approximation
h = sum([S.OrthBase[i] * Coeffs[i] for i in range(len(S.OrthBase))])
T = RationalApprox(S)
g = T.RatLSQ(5, 5, f)

print(h)
print(g)
```

CHAPTER

SIXTEEN

INTERPOLATION

16.1 Lagrange interpolation

Suppose that a list of $m+1$ points $\{(x_0, y_0), \ldots, (x_m, y_m)\}$ in $\mathbb{R}^2$ is given, such that $x_i \neq x_j$, if $i \neq j$. Then

$$p(\mathbf{x}) = \sum_{i=0}^{m} y_i \ell_i(\mathbf{x}),$$

is a polynomial of degree *m* which passes through all the given points. Here

$$\ell_i(\mathbf{x}) = \prod_{j \neq i} \frac{\mathbf{x} - x_j}{x_i - x_j},$$

are *Lagrange Basis Polynomials*.

This procedure can be extended to multivariate case as well.

Suppose that a list of points $\{x_1, \ldots, x_\rho\}$ in $\mathbb{R}^n$ and a list of corresponding values $\{y_1, \ldots, y_\rho\}$ are given. Let's denote by $\mathbf{X}$ the tuple $(\mathbf{X}_1, \ldots, \mathbf{X}_n)$ of variables and $\mathbf{X}_1^{e_1} \cdots \mathbf{X}_n^{e_n}$ by $\mathbf{X}^{\mathbf{e}}$, where $\mathbf{e} = (e_1, \ldots, e_n)$.

If for some $m > 0$, we have $\rho = \binom{m+n}{m}$, then the number of given points matches the number of monomials of degree at most *m* in the polynomial basis. Denote the exponents of these monomials by $\mathbf{e}_i$, $i = 1, \ldots, \rho$ and let

$$D = \begin{pmatrix} x_1^{\mathbf{e}_1} & \ldots & x_1^{\mathbf{e}_\rho} \\ \vdots & & \vdots \\ x_\rho^{\mathbf{e}_1} & \ldots & x_\rho^{\mathbf{e}_\rho} \end{pmatrix}$$

and for $1 \leq j \leq \rho$:

$$D_j = \begin{pmatrix} x_1^{\mathbf{e}_1} & \ldots & x_1^{\mathbf{e}_\rho} \\ \vdots & \vdots & \vdots \\ x_{j-1}^{\mathbf{e}_1} & \ldots & x_{j-1}^{\mathbf{e}_\rho} \\ \mathbf{X}^{\mathbf{e}_1} & \ldots & \mathbf{X}^{\mathbf{e}_\rho} \\ x_{j+1}^{\mathbf{e}_1} & \ldots & x_{j+1}^{\mathbf{e}_\rho} \\ \vdots & \vdots & \vdots \\ x_\rho^{\mathbf{e}_1} & \ldots & x_\rho^{\mathbf{e}_\rho} \end{pmatrix}.$$

Then the polynomial

$$p(\mathbf{X}) = \sum_{i=1}^{\rho} y_i \frac{|D_i|}{|D|},$$

interpolates the given list of points and their corresponding values. Here $|M|$ denotes the determinant of *M*.

The above procedure is implemented in the `Interpolation` module. The following code provides an example in 2 dimensional case:

```
from sympy import *
from pyProximation import Interpolation
# define symbolic variables
x = Symbol('x')
y = Symbol('y')
# initiate the interpolation instance
Inter = Interpolation([x, y], 'sympy')
# list of points
points = [(-1, 1), (-1, 0), (0, 0), (0, 1), (1, 0), (1, -1)]
# corresponding values
values = [-1, 2, 0, 2, 1, 0]
# interpolate
p = Inter.Interpolate(points, values)
# print the result
print(p)
```

16.2 L^2-approximation with discrete measures

Suppose that $\mu = \sum_1^n \delta_{x_i}$ is a measure with *n*-points in its support. Then the orthogonal system of polynomials consists of at most *n+1* polynomials. Approximation with these *n+1* polynomials is essentially same as interpolation:

```
# symbolic variable
x = Symbol('x')
# function to be approximated
g = sin(x)*exp(sin(x)*x)#x*sin(x)
# its numerical equivalent
g_ = lambdify(x, g, 'numpy')
# number of approximation terms
n = 6
# half interval length
l = 3.1
# interpolation points and values
Xs = [[-3], [-2], [-1], [0], [1], [2], [3]]
Ys = [g_(Xs[i][0]) for i in range(7)]
# a discrete measure
supp = {-3:1, -2:1, -1:1, 0:1, 1:1, 2:1, 3:1}
M = Measure(supp)
# orthogonal system
S = OrthSystem([x], [(-l, l)])
# link the measure
S.SetMeasure(M)
# polynomial basis
B = S.PolyBasis(n)
# link the basis to the orthogonal system
S.Basis(B)
# form the orthonormal basis
S.FormBasis()
# calculate coefficients
cfs = S.Series(g)
# orthogonal approximation
aprx = sum([S.OrthBase[i]*cfs[i] for i in range(len(B))])
# interpolate
```

(continues on next page)

(continued from previous page)

```
Intrp = Interpolation([x])
intr = Intrp.Interpolate(Xs, Ys)
print(intr)
```

CHAPTER

SEVENTEEN

CODE DOCUMENTATION

class pyProximation.base.**Foundation**

This class contains common features of all modules.

DetSymEnv()

Returns `['sympy']` when sympy is available.

class pyProximation.measure.**Measure**(*dom*, *w=None*)

An instance of this class is a measure on a given set *supp*. The support is either

- a python variable of type *set*, or
- a list of tuples which represents a box in euclidean space.

Initializes a measure object according to the inputs:

- ***dom* must be either**
  - a list of 2-tuples
  - a non-empty dictionary
- ***w* must be a**
  - a function if *dom* defines a region
  - left blank (None) if *dom* is a dictionary

boxCheck(*B*)

Checks the structure of the box *B*. Returns *True* id *B* is a list of 2-tuples, otherwise it returns *False*.

check(*dom*, *w*)

Checks the input types and their consistency, according to the *__init__* arguments.

integral(*f*)

Returns the integral of *f* with respect to the currwnt measure over the support.

measure(*S*)

Returns the measure of the set *S*. *S* must be a list of 2-tuples.

norm(*p*, *f*)

Computes the norm-*p* of the *f* with respect to the current measure.

sample(*num*)

Samples from the support according to the measure.

class pyProximation.orthsys.**OrthSystem**(*variables*, *var_range*, *env='sympy'*)

`OrthogonalSystem` class produces an orthogonal system of functions according to a suggested basis of functions and a given measure supported on a given region.

This basically performs a 'Gram-Schmidt' method to extract the orthogonal basis. The inner product is obtained by integration of the product of functions with respect to the given measure (more accurately, the distribution).

To initiate an instance of this class one should provide a list of symbolic variables *variables* and the range of each variable as a list of lists `var_range`.

To initiate an orthogonal system of functions, one should provide a list of symbolic variables `variables` and the range of each these variables as a list of lists `var_range`.

Basis(*base_set*)

To specify a particular family of function as a basis, one should call this method with a list `base_set` of linearly independent functions.

FormBasis()

Call this method to generate the orthogonal basis corresponding to the given basis via `Basis` method. The result will be stored in a property called `OrthBase` which is a list of function that are orthogonal to each other with respect to the measure `measure` over the given range `Domain`.

FourierBasis(*n*)

Generates a Fourier basis from variables consisting of all sin & cos functions with coefficients at most *n*.

PolyBasis(*n*)

Generates a polynomial basis from variables consisting of all monomials of degree at most `n`.

Series(*f*)

Given a function *f*, this method finds and returns the coefficients of the series that approximates *f* as a linear combination of the elements of the orthogonal basis.

SetMeasure(*M*)

To set the measure which the orthogonal system will be computed, simply call this method with the corresponding distribution as its parameter *dm*; i.e, the parameter is *d(m)* where *m* is the original measure.

SetOrthBase(*base*)

Sets the orthonormal basis to be the given *base*.

TensorPrd(*Bs*)

Takses a list of symbolic bases, each one a list of symbolic expressions and returns the tensor product of them as a list.

inner(*f*, *g*)

Computes the inner product of the two parameters with respect to the measure `measure`.

project(*f*, *g*)

Finds the projection of `f` on `g` with respect to the inner product induced by the measure `measure`.

pyProximation.rational.**RationalApprox**

alias of *RationalAprox*

class pyProximation.rational.**RationalAprox**(*orth*)

`RationalAprox` calculates a rational approximation for a given function. `RationalApprox` is the preferred public alias; this class name is retained for backward compatibility. It takes one argument *orth* which is an instance of `OrthSystem` and does all the computations in the scope of this object.

RatLSQ(*m*, *n*, *f*)

Calculates a rational function :math:` rac{p}{q}` where p, q consists of the first m, n elements of the orthonormal basis $||f - racpq||_2$ is minimized.

class pyProximation.interpolation.**Interpolation**(*var*, *env='sympy'*)

The `Interpolation` class provides polynomial interpolation routines in multi variate case.

var is the list of symbolic variables and *env* is the the symbolic tool.

Delta(*idx=-1*)

Construct the matrix corresponding to *idx*'th point, if *idx>0* Otherwise returns the discriminant.

Interpolate(*points*, *vals*)

Takes a list of points *points* and corresponding list of values *vals* and return the interpolant.

Since in multivariate case, there is a constraint on the number of points, it checks for the valididty of the input. In case of failure, describes the type of error occured according to the inputs.

MinNumPoints()

Returns the minimum number of points still required.

Monomials()

Generates the minimal set of monomials for interpolation.

CHAPTER

EIGHTEEN

REVISION HISTORY

Version 1.2.5 (Mar 12, 2026)

- Expanded documentation from SDP-only emphasis to a unified POP guide covering SDP, geometric programming, and SONC relaxations.
- Added new chapters for architecture, group-ring foundations, optimization problem representation, geometric relaxations, SONC relaxations, and runnable examples.
- Added method-selection guidance, dependency matrix, and solver troubleshooting notes.
- Added explicit theory-to-code mapping for constrained SONC families (Section 3 style equations) and geometric lower-bound construction (Section 4 equation mapping).
- Expanded API documentation coverage in Sphinx for `program`, `grouprings`, `geometric`, and `sonc` modules.
- Updated Sphinx configuration for modern toolchain compatibility and warning-free documentation builds.

Version 1.2.2 (Mar 15, 2017)

- Serialization and pickling: Saving the latest state of the program on break which can be retrieved later and resume.

Version 1.2.1 (Mar 2, 2017)

- Removed dependency on `joblib` for multiprocessing.

Version 1.2.0 (Jan 5, 2017)

- LaTeX representation of Irene's objects.
- `SDRelaxSol` can be called as an iterable.
- Default objective function is set to a `sympy` object for truncated moment problems.

Version 1.1.0 (Dec 25, 2016 - Merry Christmas)

- Extracting minimizers via `SDRelaxSol.ExtractSolution()` and help of `scipy`,
- Extracting minimizers implementing Lasserre-Henrion algorithm,
- Adding `SDPRelaxations.Probability` and `SDPRelaxations.PSDMoment` to give more flexibility over moments and enables rational minimization.
- SOS decomposition implemented.
- `__str__` method for `SDPRelaxations`.
- Using *pyOpt* as the external optimizer.
- More benchmark examples.

Version 1.0.0 (Dec 07, 2016)

- Initial release (Birth of Irene)

CHAPTER

NINETEEN

TO DO

Based on the current implementation, the followings seems to be implemented/modified:

- Reduce dependency on SymPy.
- Include sdp solvers installation (subject to copyright limitations).
- Error handling for CSDP and SDPA failure.

19.1 Done

The following to-dos were implemented:

- Extract solutions (at least for polynomials)- in v.1.1.0.
- SOS decomposition- in v.1.1.0.
- Write a `__str__` method for `SDPRelaxations` printing- in v.1.1.0.
- Write a LaTeX method- in v.1.2.0.
- Keep track of original expressions before reduction- in v.1.2.0.
- Removed dependency on `joblib`- in v.1.2.1.
- Save the current status on break and resume later- in v.1.2.2.
- Windows support- in v.1.2.3.

CHAPTER

TWENTY

APPENDIX

20.1 pyProximation

pyProximation is a companion library for function approximation in Hilbert spaces. Full documentation is included in this manual — see the *pyProximation* section.

20.2 pyOpt

pyOpt is a Python-based package for formulating and solving nonlinear constrained optimization problems in an efficient, reusable and portable manner. It is an open-source software distributed under the terms of the GNU Lesser General Public License.

***pyOpt* provides unified interface to the following nonlinear optimizers:**

- SNOPT - Sparse NOlinear OPTimizer
- NLPQL - Non-Linear Programming by Quadratic Lagrangian
- NLPQLP - NonLinear Programming with Non-Monotone and Distributed Line Search
- FSQP - Feasible Sequential Quadratic Programming
- SLSQP - Sequential Least Squares Programming
- PSQP - Preconditioned Sequential Quadratic Programming
- ALGENCAN - Augmented Lagrangian with GENCAN
- FILTERSD
- MMA - Method of Moving Asymptotes
- GCMMA - Globally Convergent Method of Moving Asymptotes
- CONMIN - CONstrained function MINimization
- MMFD - Modified Method of Feasible Directions
- KSOPT - Kreisselmeier–Steinhauser Optimizer
- COBYLA - Constrained Optimization BY Linear Approximation
- SDPEN - Sequential Penalty Derivative-free method for Nonlinear constrained optimization
- SOLVOPT - SOLver for local OPTimization problems
- ALPSO - Augmented Lagrangian Particle Swarm Optimizer
- NSGA2 - Non Sorting Genetic Algorithm II
- ALHSO - Augmented Lagrangian Harmony Search Optimizer

- MIDACO - Mixed Integer Distributed Ant Colony Optimization

20.2.1 Basic usage:

pyOpt is design to solve general constrained nonlinear optimization problems:

$$\begin{cases} \min & f(x) \\ \text{Subject to} & \\ & g_j(x) = 0 \qquad j = 1, \ldots, m_e \\ & g_j(x) \leq 0 \qquad j = m_e + 1, \ldots, m \\ & l_i \leq x_i \leq u_i \quad i = 1, \ldots, n, \end{cases}$$

where:

- x is the vector of design variables
- $f(x)$ is a nonlinear function
- $g(x)$ is a linear or nonlinear function
- n is the number of design variables
- m_e is the number of equality constraints
- m is the total number of constraints (number of equality constraints: $m_i = m - m_e$).

The following is a pseudo-code demonstrating the basic usage of `pyOpt`:

```
# General Objective Function Template:
def obj_fun(x, *args, **kwargs):
        """
        f: objective value
        g: array (or list) of constraint values
        fail: 0 for successful function evaluation, 1 for unsuccessful function␣
↪evaluation (test must be provided by user)
        If the Optimization problem is unconstrained, g must be returned as an empty␣
↪list or array: g = []
        Inequality constraints are handled as `<=`.
        """
        fail = 0
        f = function(x,*args,**kwargs)
        g = function(x,*args,**kwargs)

        return f,g,fail

# Instantiating an Optimization Problem:
opt_prob = Optimization('name', obj_fun)
# Assigning Objective:
opt_prob.addObj('name', value=0.0, optimum=0.0)
# Single Design variable:
opt_prob.addVar('name', type='c', value=0.0, lower=-inf, upper=inf, choices=listochoices)
# A Group of Design Variables:
opt_prob.addVarGroup('name', numerinGroup, type='c', value=value, lower=lb, upper=up,␣
↪choices=listochoices)
# where `value`, `lb`, `ub` (float or int or list or 1Darray).
# and supported Types are 'c': continuous design variable;
# `i`: integer design variable;
```

(continues on next page)

(continued from previous page)

```
# `d`: discrete design variable (based on choices, e.g.: list/dict of materials).
# Assigning Constraints:
## Single Constraint:
opt_prob.addCon('name', type='i', lower=-inf, upper=inf, equal=0.0)
## A Group of Constraints:
opt_prob.addConGroup('name', numberinGroup, type='i', lower=lb, upper=up, equal=eq)
# where `lb`, `ub`, `eq` are (float or int or list or 1Darray).
# and supported types are
# `i` - inequality constraint;
# `e` - equality constraint.

# Instantiating an Optimizer (e.g.: Snopt):
opt = pySNOPT.SNOPT()
# Solving the Optimization Problem:
opt(opt_prob, sens_type='FD', disp_opts=False, sens_mode='', *args, **kwargs)
# Output:
print opt_prob
```

For more details, see pyOpt documentation.

20.3 LaTeX support

There is a method implemented for every class of Irene module that generates a LaTeX code related to the object. This code can be retrieved by calling `LaTeX` function on the instance.

Calling `LaTeX` on a

- `SDPRelaxations` instance returns the code which demonstrates user entered optimization problem.
- `SDRelaxSol` instance returns the code for the moment matrix of the solution.
- `Mom` instance (moment object) returns the LaTeX code of the object.
- `sdp` instance returns the code for the SDP.

Moreover, if `LaTeX` function is called on a `SymPy` object it calls `sympy.latex` function and returns the output.

The `LaTeX` function is a member of `Irene.base` module which calls `__latex__` method of its classes.

CHAPTER

TWENTYONE

INDICES AND TABLES

- genindex
- modindex
- search

BIBLIOGRAPHY

[GIKM] M. Ghasemi, M. Infusino, S. Kuhlmann and M. Marshall, *Truncated Moment Problem for unital commutative real algebras*, to appear.

[JBL] J-B. Lasserre, *Global optimization with polynomials and the problem of moments*, SIAM J. Optim. 11(3) 796-817 (2000).

[JNie] J. Nie, *The A-Truncated K-Moment Problem*, Found. Comput. Math., Vol.14(6), 1243-1276 (2014).

[HL] D. Henrion and J-B. Lasserre, *Detecting Global Optimality and Extracting Solutions in GloptiPoly*, Positive Polynomials in Control, LNCIS 312, 293-310 (2005).

[MJXY] M. Jamil, Xin-She Yang, *A literature survey of benchmark functions for global optimization problems*, IJMMNO, Vol. 4(2), 2013.

PYTHON MODULE INDEX

i

Irene.base, 75
Irene.geometric, 95
Irene.grouprings, 89
Irene.program, 81
Irene.relaxations, 75
Irene.sdp, 79
Irene.sonc, 95

p

pyProximation.base, 113
pyProximation.interpolation, 115
pyProximation.measure, 113
pyProximation.orthsys, 113
pyProximation.rational, 114